431 Classes 22-24

thomaselove.github.io/431

2020-11-12

Agenda for Classes 22-24

- Power / Sample Size Decisions and The March of Science
- Multiple Regresssion using the `dm431` data
  - Using `df_stats` to get `favstats` for multiple variables at once
  - Using the `naniar` package to identify and summarizing missingness
  - Complete Cases and Simple imputation to deal with missingness
  - Partitioning our data into training/test samples
  - Outcome transformation: what to consider
  - Assessing the fit in the sample where we build the model
    - Using `tidy` to describe model coefficients
    - Using `glance` to study fit quality
    - Using `augment` to obtain predicted values and residuals
    - Residual plots to check assumptions with `plot` and with `ggplot2`
  - Testing the model in new data (a holdout sample)
    - Assessing the predictions and prediction errors
    - Back-transformation and MAPE, root MSPE and max error

Today's Setup

```
knitr::opts_chunk$set(comment = NA)
options(dplyr.summarise.inform = FALSE)

library(naniar) # new
library(simputation) # new
library(car) # for boxCox
library(GGally) # for ggpairs
library(ggrepel) # help with residual plots
library(equatiomatic) # help with equation extraction
library(knitr)
library(janitor)
library(magrittr)
library(patchwork)
library(broom)
library(tidyverse)

theme_set(theme_bw())
```

On Power / Sample Size Decisions and the March of Science

The March of Science and Power Calculations

I mentioned last time that the most common scenario was to identify:

- desired significance level α
- desired power (1 - β)

and the details of the plan in terms of what comparison is to be made, and how will the data be collected to support that comparison.

This will then permit the calculation of a minimum necessary sample size to achieve these desires.

I also mentioned that $\alpha = 0.05$ and $\beta = 0.2$ were the most common selections.

- Neither 95% confidence nor 80% power is a magical choice.
- Anything below 80% power will be hard to justify in real work.

A useful metaphor?

Sometimes I like to think of science as a march towards a destination.

Actually, I suppose it's an infinitely long march towards an ever-receding destination, but let's leave the philosophy out of it for a moment.

Suppose, for example, that we're trying to make a meaningful change in the world, perhaps to treat an infection.

What we're trying to do is related to where we are in the March of Science.

Early vs. Late in the March of Science

In **early** work, we're focused more on discovery than making final decisions.

- We don't have a lot of past experience, so we bring little relevant data to the table.
- We're (often) most concerned about discovering new possibilities, and we don't have a very clear sense of where to go next.
- We're (often) less concerned about false starts than we are about missed opportunities.

In **late** work, we're focused more on making a decision about how to treat.

- We have a fair amount of relevant history to draw on, sometimes quite detailed.
- We're more concerned about testing the limits of our current knowledge than we are about missing opportunities to consider a new pathway.
- We're often concerned about doing harm if we implement the strategy that looks most promising.

How does this relate to power and significance?

If we treat our sample size and study design as fixed strategies, then there is a tradeoff between:

- reducing α, the rate of Type I error (increasing our confidence) and
- reducing β, the rate of Type II error (increasing our power)

Suppose we are testing a new treatment for some condition.

- A Type I error means we conclude this treatment is helpful, when it actually isn't.
- A Type II error means we conclude this treatment is not helpful, when it actually is.

Early Work: Power and Sample Size

In early work, we are searching for treatments of promise, and our initial study will inevitably not be the last word on the subject, but rather will be followed up by confirmatory studies. In such a setting, it is often the case that:

- We're not so concerned about getting results that cause us to continue to explore a treatment that doesn't actually do what we need it to do.
- We're really concerned about ruling out a treatment that is promising before we should.

This implies we should prioritize reducing Type II error rates (we want more power to detect small but real effects, even if this means we will occasionally identify something as promising when it isn't.)

This means setting lower confidence levels and higher power levels, potentially, than the standard 95% confidence and 80% power.

Late Work: Power and Sample Size

In late work, we have already identified promising treatments, and we are trying to confirm those results. The current study may actually be the last word on the subject, and we want to be sure we do no harm.

- We're very concerned about getting results that cause us to continue to explore a treatment that doesn't actually do what we need it to do.
- We're less concerned about ruling out a treatment that is promising but doesn't actually work.

This implies we should prioritize reducing Type I error rates (we want greater confidence, even at the expense of power, that the effect we claim based on past data holds up.)

This kind of confirmatory work is usually well suited to studies set up with higher confidence (perhaps 95% or 99% or more) and lower power (80% is the minimum I would recommend) against reasonable alternatives.

Conclusions

1. If you're early in the March of Science (perhaps just one pilot study has been done) then I would emphasize Type II error (power) more than usual, perhaps pushing required power to 90%, at least for a reasonably substantial "minimum scientifically important difference" δ.
2. If you're late in the March of Science (perhaps confirming the results of multiple prior studies) then I would be happier with 80% power and higher levels of confidence.
3. If you're in the middle, trading off Type I and Type II error is worth some thought, but I'd never recommend being under 80% power for an effect that matters.
4. If it's feasible to run a study large enough to have strong performance on both α and β, that's obviously ideal. Typically that doesn't happen in early work.

Multiple Regression with the `dm431` data

Focus on Four Variables (+ Subject)

Suppose we want to consider predicting the `a1c` values of our diabetes subjects now, based on these three predictors:

- `a1c_old`: subject's Hemoglobin A1c (in %) two years ago
- `age`: subject's age in years
- `income`: median income of subject's home neighborhood (3 categories)

```
dm1 <- dm431 %>%
  select(a1c, a1c_old, age, income, subject)
```

What roles will these variables play?

`a1c` is our outcome, which we'll predict using three models . . .

1. Model 1: Use `a1c_old` alone to predict `a1c`
2. Model 2: Use `a1c_old` and `age` together to predict `a1c`
3. Model 3: Use `a1c_old`, `age`, and `income` together to predict `a1c`

Summarizing the dm1 tibble

```r
summary(dm1)
```

```
      a1c             a1c_old             age
 Min.   : 4.300   Min.   : 4.200   Min.   :31.00
 1st Qu.: 6.500   1st Qu.: 6.500   1st Qu.:51.00
 Median : 7.300   Median : 7.300   Median :57.00
 Mean   : 7.884   Mean   : 7.712   Mean   :56.14
 3rd Qu.: 8.600   3rd Qu.: 8.400   3rd Qu.:62.00
 Max.   :16.700   Max.   :16.300   Max.   :70.00
 NA's   :3        NA's   :14
             income      subject
 Below_30K       :146   Length:431
 Between_30-50K  :171   Class :character
 Higher_than_50K:110   Mode  :character
 NA's            :  4
```

df_stats to get favstats on multiple quantities

Suppose we want favstats results on all 3 quantitative variables.

```
dm1 %>%
  mosaicCore::df_stats(~ a1c + a1c_old + age) %>%
  rename(na = missing) %>% kable(dig = 2)
```

response	min	Q1	median	Q3	max	mean	sd	n	na
a1c	4.3	6.5	7.3	8.6	16.7	7.88	2.03	428	3
a1c_old	4.2	6.5	7.3	8.4	16.3	7.71	1.77	417	14
age	31.0	51.0	57.0	62.0	70.0	56.14	8.41	431	0

- The df_stats function is part of the mosaicCore package.
- Either use library(mosaic) to make df_stats() available, or specify it with mosaicCore::df_stats().

Identifying Missingness and the `naniar` package

What will we do about missing data?

```
dm1 %>%
  summarize(across(everything(), ~ sum(is.na(.)))) %>%
  kable(dig = 0)
```

a1c	a1c_old	age	income	subject
3	14	0	4	0

- We're missing 3 values of `a1c`, our outcome
- and 14 values of `a1c_old`, a predictor (Models 1-3)
- and 4 values of `income`, another predictor (Model 3)

But what if we have other questions, like:

- How many observations are missing at least one of these variables?
- How many subjects (cases) are missing multiple variables?

Counting missingness by subject with `naniar`

The `naniar` package provides several useful functions for identifying and summarizing missingness which work within a tidy workflow.

`miss_case_table()` for instance, provides a summary table describing the number of subjects missing 0, 1, 2, ... of the variables in our tibble.

```
miss_case_table(dm1)
```

```
# A tibble: 3 x 3
  n_miss_in_case n_cases pct_cases
           <int>   <int>     <dbl>
1              0     412    95.6
2              1      17     3.94
3              2       2     0.464
```

So, there are 17 subjects missing one variable, and 2 missing two.

Can we identify these cases?

miss_case_summary lists missingness for each subject

```r
miss_case_summary(dm1)
```

```
# A tibble: 431 x 3
    case n_miss pct_miss
   <int>  <int>    <dbl>
 1   280      2       40
 2   319      2       40
 3     3      1       20
 4     7      1       20
 5    16      1       20
 6    20      1       20
 7    41      1       20
 8    49      1       20
 9    67      1       20
10    82      1       20
# ... with 421 more rows
```

Can we summarize missingness by variable?

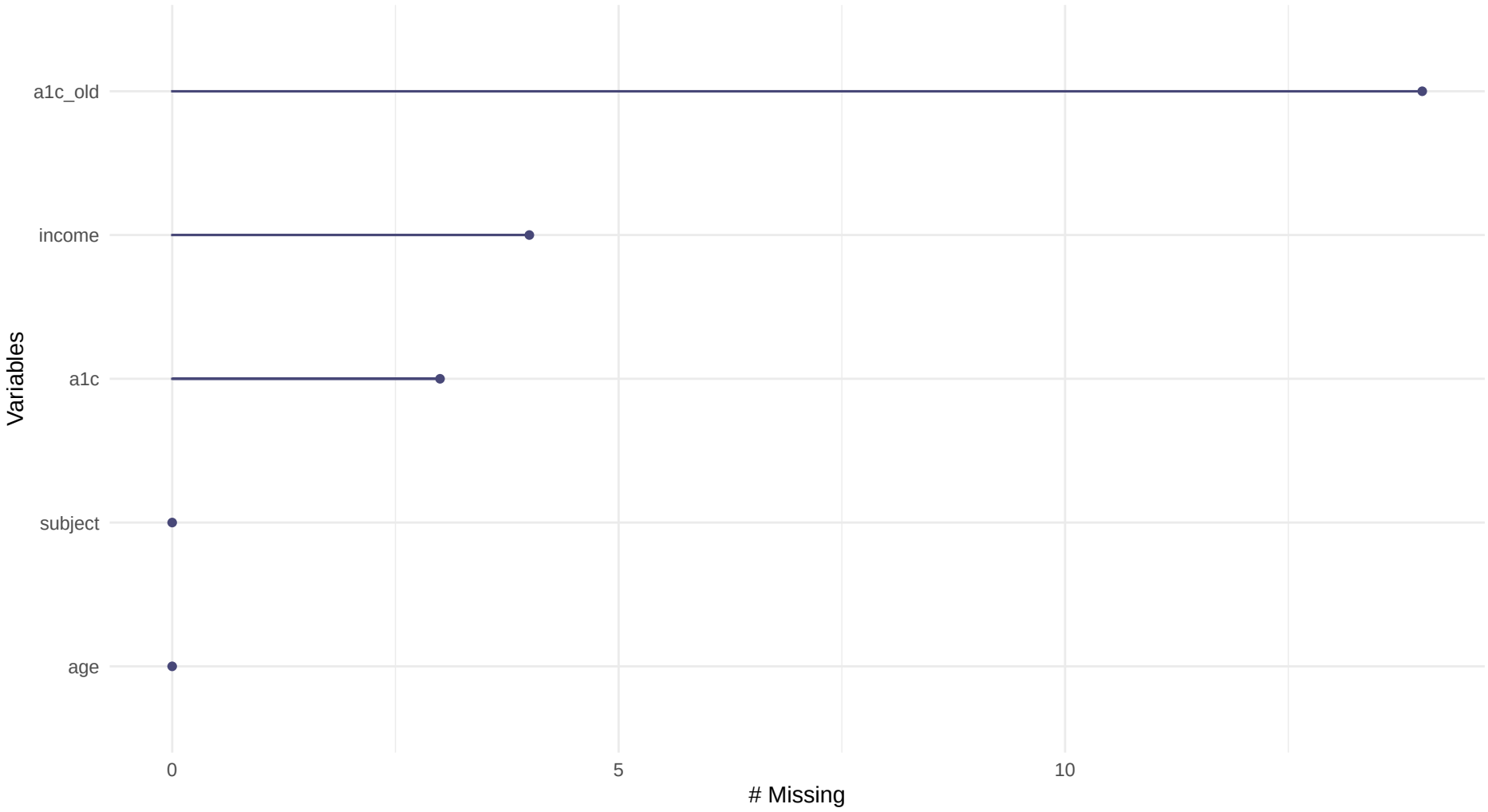
```r
miss_var_summary(dm1)
```

```
# A tibble: 5 x 3
  variable n_miss pct_miss
  <chr>     <int>    <dbl>
1 a1c_old      14    3.25
2 income        4    0.928
3 a1c           3    0.696
4 age           0    0
5 subject       0    0
```

There's a `miss_var_table()` function, too, if that's useful.

nanIar also has helpers for plots

```
gg_miss_var(dm1)
```



Option 1: Complete Cases Only

We might assume that all of our missing values are Missing Completely At Random (MCAR) and thus that we can safely drop all observations with missing data from our data set.

```
dm1_cc <- dm1 %>% filter(complete.cases(.))

nrow(dm1)
```

```
[1] 431
```

```
nrow(dm1_cc)
```

```
[1] 412
```

- For today, I want to use the same observations in each of my 3 models.
- So we would drop 19 subjects, and fit models with the 412 subjects who have complete data on all four variables.

Simple Imputation with the `simputation` package

Option 2: Simple Imputation

Suppose I don't want to impute the outcome. I think people missing my outcome shouldn't be included in my models.

- We'll drop the 3 observations missing `a1c` from our data set.

Perhaps I'd be OK with assuming the missing values of `income` or `a1c_old` are MAR (so that we could use variables in our data to predict them.)

- This would allow us to use imputation methods to "fill in" or "impute" missing predictor values so that we can still use all of the other 428 subjects in our models.
- The `simputation` package provides a straightforward method to do this, while maintaining a tidy workflow.
- There are dangers in assuming everything is MCAR, so this looks helpful (MAR is a lesser assumption) but it introduces the issue of "creating" data where it didn't exist.

Simple Imputation of Missing a1c_old Values

We could use a robust linear model method to impute our quantitative a1c_old values on the basis of age, which is missing no observations in common with a1c_old (in fact, age is missing no observations.)

```
tempA <- impute_rlm(dm1, a1c_old ~ age)

tempA %>% miss_var_summary()
```

```
# A tibble: 5 x 3
  variable n_miss pct_miss
  <chr>     <int>    <dbl>
1 income        4    0.928
2 a1c           3    0.696
3 a1c_old       0    0
4 age           0    0
5 subject       0    0
```

Simple Imputation of Missing `income` Values

We could use a decision tree (CART) method to impute our missing categorical `income` values, also on the basis of age.

```
tempB <- impute_cart(dm1, income ~ age)

tempB %>% miss_var_summary()
```

```
# A tibble: 5 x 3
  variable n_miss pct_miss
  <chr>     <int>    <dbl>
1 a1c_old      14    3.25
2 a1c           3    0.696
3 age           0    0
4 income        0    0
5 subject       0    0
```

Chaining our Simple Imputations

Or we could put all of our imputations together in a chain.

- In 431, I encourage you to try `rlm` for imputing quantitative variables, and `cart` for categorical variables.
- Were I imputing a binary categorical variable, I would present it as a factor to `impute_cart`.

```
dm1_imp <- dm1 %>%
  filter(complete.cases(a1c, subject)) %>%
  impute_rlm(a1c_old ~ age) %>%
  impute_cart(income ~ age + a1c_old)
```

- I imputed `a1c_old` using age and then imputed `income` using both age and `a1c_old`.

What is the result?

Summary of imputed tibble

dm1_imp has 428 observations (since we dropped the 3 subjects with missing a1c: our *outcome*) but no missing values left.

```
dm1_imp %>% summary()
```

```
      a1c            a1c_old            age
 Min.   : 4.300   Min.   : 4.200   Min.   :31.00
 1st Qu.: 6.500   1st Qu.: 6.500   1st Qu.:50.75
 Median : 7.300   Median : 7.300   Median :57.00
 Mean   : 7.884   Mean   : 7.711   Mean   :56.09
 3rd Qu.: 8.600   3rd Qu.: 8.400   3rd Qu.:62.00
 Max.   :16.700   Max.   :16.300   Max.   :70.00
             income      subject
 Below_30K      :148   Length:428
 Between_30-50K :171   Class :character
 Higher_than_50K:109   Mode  :character
```

Two approaches for dealing with missing data

1. We could assume MCAR for all variables, and then work with the complete cases (n = 412) in `dm1_cc`.
2. We could assume MAR for the predictors, and work with the simply imputed (n = 428) in `dm1_imp`

Neither of these, as it turns out, will be 100% satisfactory, but for now, we'll compare the impact of these two approaches on the results of our models.

OK. We'll do the complete case analysis now, and return to the imputed data later.

How will we decide which of the models is "best"?

Our goal is accurate prediction of `a1c` values.

Which of these models gives us the "best" result?

1. Model 1: Use `a1c_old` alone to predict `a1c`
2. Model 2: Use `a1c_old` and `age` together to predict `a1c`
3. Model 3: Use `a1c_old`, `age`, and `income` together to predict `a1c`

and does our answer change depending on whether we start our work with the complete cases (`dm1_cc`: n = 412) or our simply imputed data (`dm1_imp`: n = 428)?

How shall we be guided by our data?

It can scarcely be denied that the supreme goal of all theory is to make the irreducible basic elements as simple and as few as possible without having to surrender the adequate representation of a single datum of experience. (A. Einstein)

- often this is reduced to "make everything as simple as possible but no simpler"

  Entities should not be multiplied without necessity. (Occam's razor)

- often this is reduced to "the simplest solution is most likely the right one"

George Box's aphorisms

On Parsimony: Since all models are wrong the scientist cannot obtain a "correct" one by excessive elaboration. On the contrary following William of Occam he should seek an economical description of natural phenomena. Just as the ability to devise simple but evocative models is the signature of the great scientist so overelaboration and overparameterization is often the mark of mediocrity.

On Worrying Selectively: Since all models are wrong the scientist must be alert to what is importantly wrong. It is inappropriate to be concerned about mice when there are tigers abroad.

- and, the most familiar version...

  ... all models are approximations. Essentially, all models are wrong, but some are useful. However, the approximate nature of the model must always be borne in mind.

431 approach: Which model is "most useful"?

1. Split the data into a development (model training) sample of about 70-80% of the observations, and a holdout (model test) sample, containing the remaining observations.
2. Develop candidate models using the development sample.
3. Assess the quality of fit for candidate models within the development sample.
4. Check adherence to regression assumptions in the development sample.
5. When you have candidates, assess them based on the accuracy of the predictions they make for the data held out (and thus not used in building the models.)
6. Select a "final" model for use based on the evidence in steps 3, 4 and especially 5.

Split the data into a model development (training) sample of about 70-80% of the observations, and a model test (holdout) sample, containing the remaining observations.

Partitioning the 412 Complete Cases

- We'll select a random sample (without replacement) of 70% of the data (70-80% is customary) for model training.
- We'll hold out the remaining 30% for model testing, using `anti_join()` to identify all `dm1_cc` subjects not in `dm1_cc_train`.

```
set.seed(20201112)

dm1_cc_train <- dm1_cc %>%
  slice_sample(prop = 0.7, replace = FALSE)

dm1_cc_test <-
  anti_join(dm1_cc, dm1_cc_train, by = "subject")

c(nrow(dm1_cc_train), nrow(dm1_cc_test), nrow(dm1_cc))
```

```
[1] 288 124 412
```

Develop candidate models using the development sample.

A look at the outcome (a1c) distribution

We'll study the outcome variable (a1c) in the development sample, to consider whether a transformation might be in order.

I did a little fancy work with the code (continues next slide)...

```
p1 <- ggplot(dm1_cc_train, aes(x = a1c)) +
  geom_histogram(binwidth = 0.5,
                 fill = "slateblue", col = "white")

p2 <- ggplot(dm1_cc_train, aes(sample = a1c)) +
  geom_qq(col = "slateblue") + geom_qq_line(col = "red")

p3 <- ggplot(dm1_cc_train, aes(x = "", y = a1c)) +
  geom_violin(fill = "slateblue", alpha = 0.3) +
  geom_boxplot(fill = "slateblue", width = 0.3,
               outlier.color = "red") +
  labs(x = "") + coord_flip()
```

A look at the outcome (a1c) distribution

Putting the plots together, and titling them meaningfully...

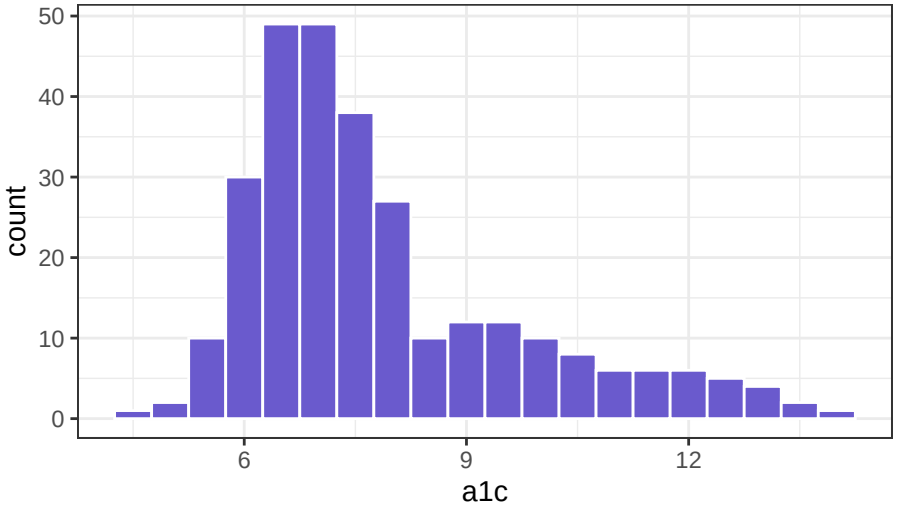
```
p1 + p2 - p3 +
  plot_layout(ncol = 1, height = c(3, 2)) +
  plot_annotation(title = "Hemoglobin A1c values (%)",
          subtitle = paste0("Model Development Sample: ",
                            nrow(dm1_cc_train),
                            " adults with diabetes"))
```

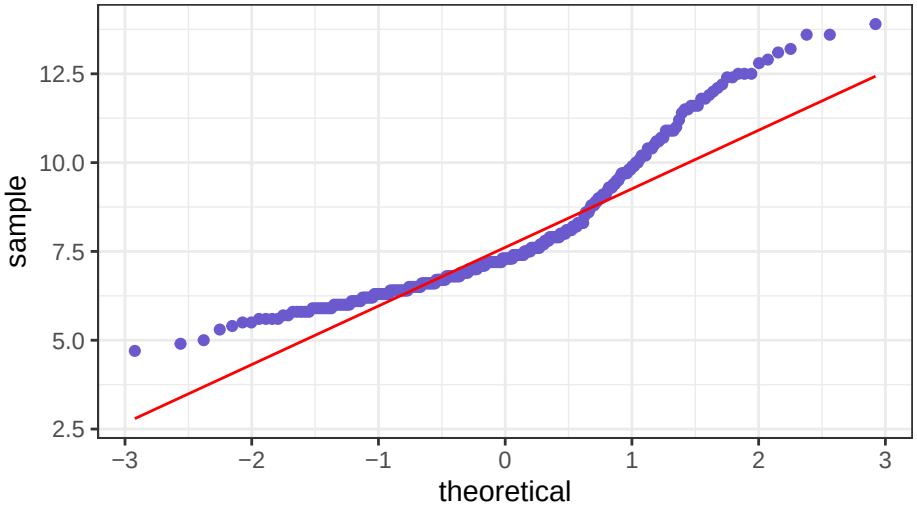
Result on the next slide...

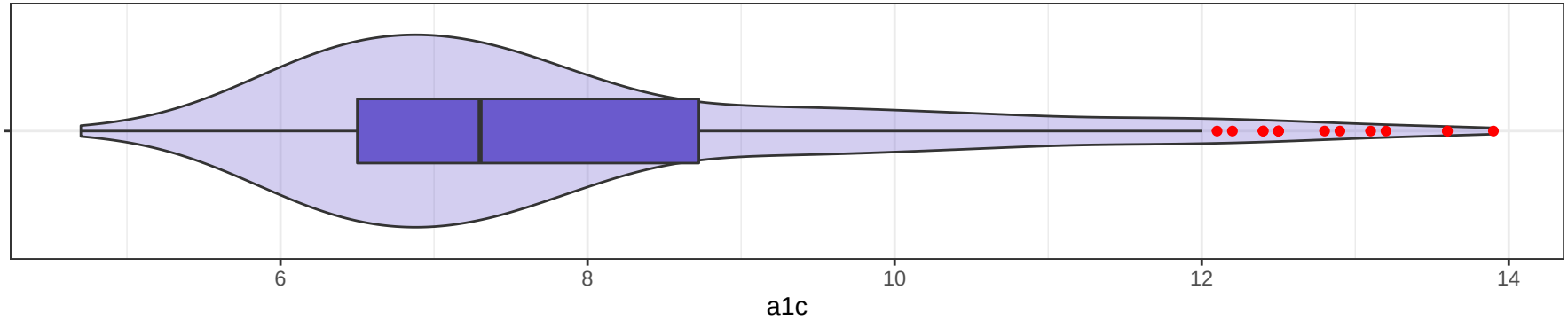
Outcome (a1c): Model Development Sample

Hemoglobin A1c values (%)

Model Development Sample: 288 adults with diabetes







Why Transform the Outcome?

We want to try to identify a good transformation for the conditional distribution of the outcome, given the predictors, in an attempt to make the linear regression assumptions of linearity, Normality and constant variance more appropriate.

Ladder of Especially Useful (and often interpretable) transformations

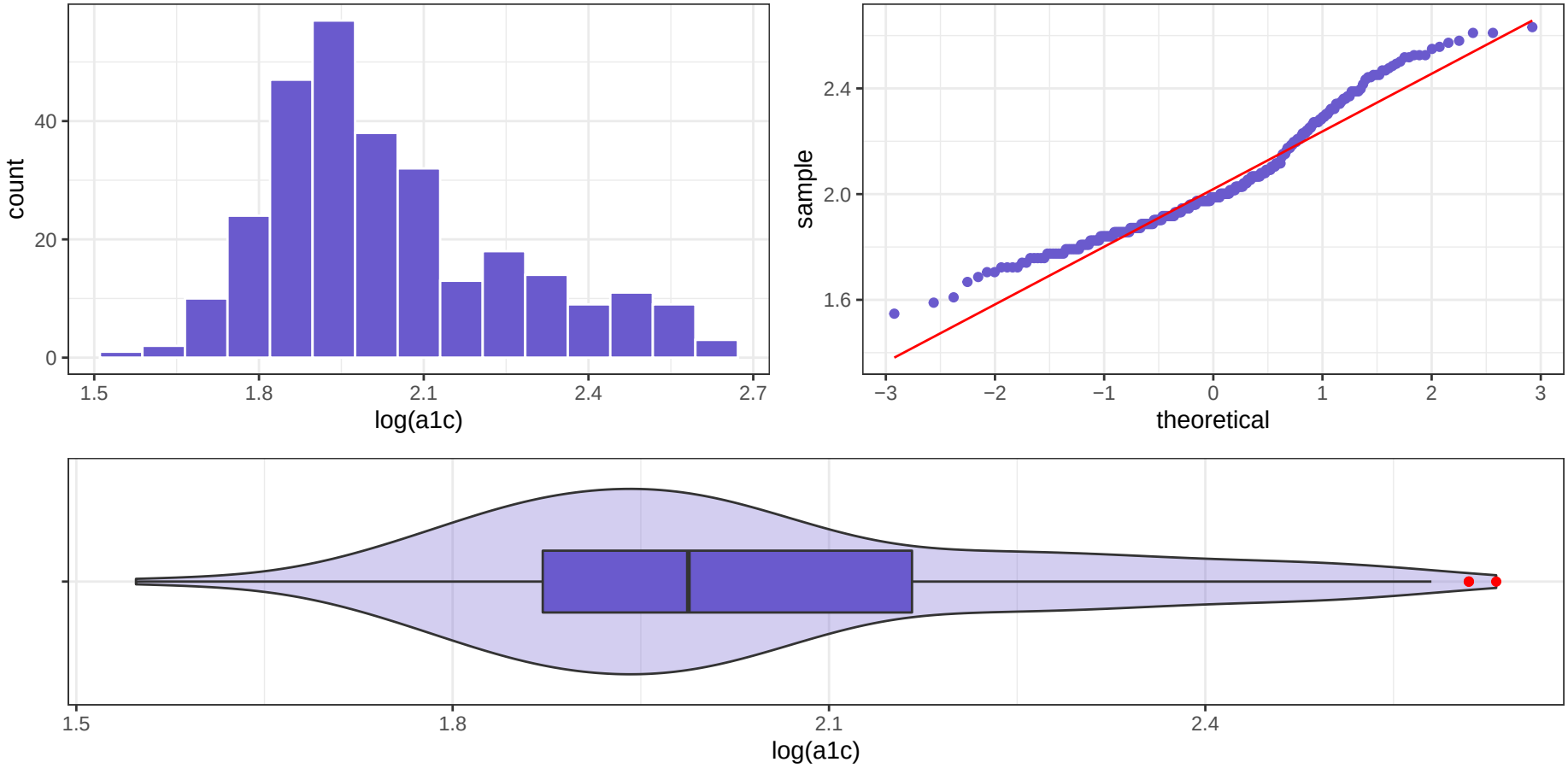
Transformation	y^2	y	$\sqrt{y}$	log(y)	$1/y$	$1/y^2$
λ	2	1	0.5	0	-1	-2

- We see some sign of right skew in the `a1c` data. Let's try a log transformation.

Consider a log transformation?

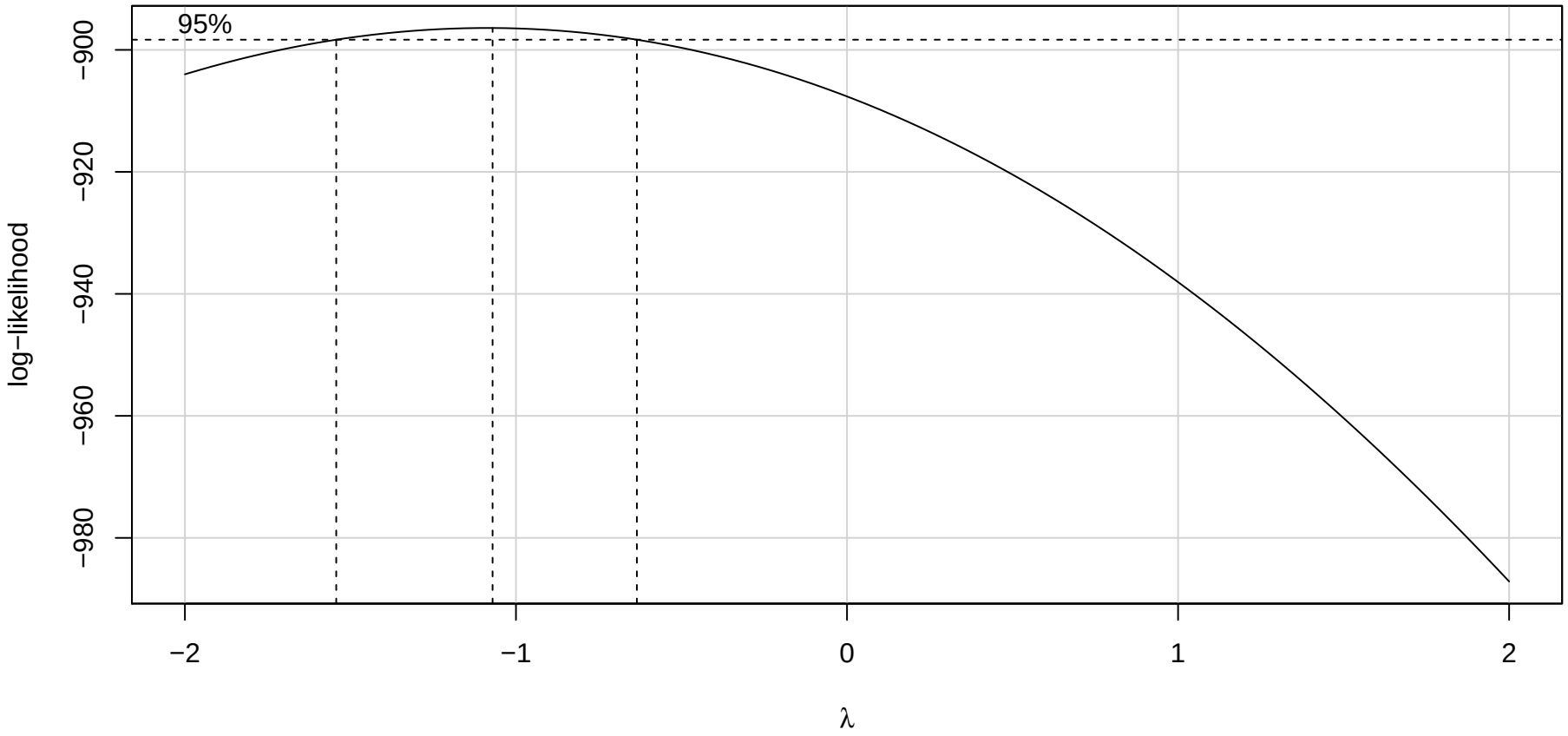
Natural Logarithm of Hemoglobin A1c

Model Development Sample: 288 adults with diabetes



Using Box-Cox to help select a transformation?

```
mod_0 <- lm(a1c ~ a1c_old + age + income,
            data = dm1_cc_train)
boxCox(mod_0)
```



Using Box-Cox to help select a transformation?

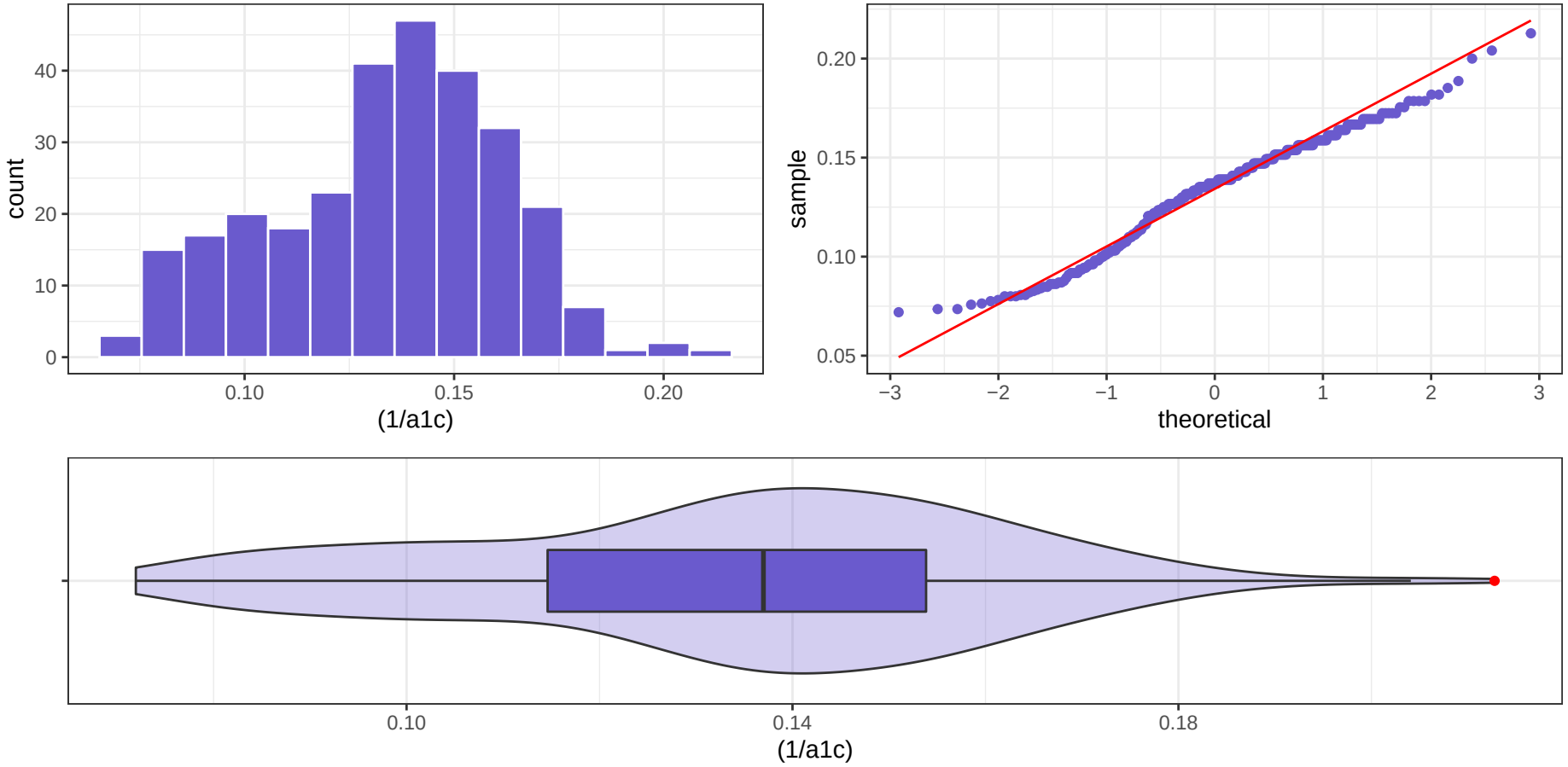
```r
summary(powerTransform(mod_0))
```

```
bcPower Transformation to Normality
   Est Power Rounded Pwr Wald Lwr Bnd Wald Upr Bnd
Y1   -1.0865          -1      -1.5404      -0.6326

Likelihood ratio test that transformation parameter is equal t
 (log transformation)
                           LRT df       pval
LR test, lambda = (0) 22.44334  1 2.1644e-06

Likelihood ratio test that no transformation is needed
                           LRT df       pval
LR test, lambda = (1) 83.31531  1 < 2.22e-16
```

Consider the inverse?

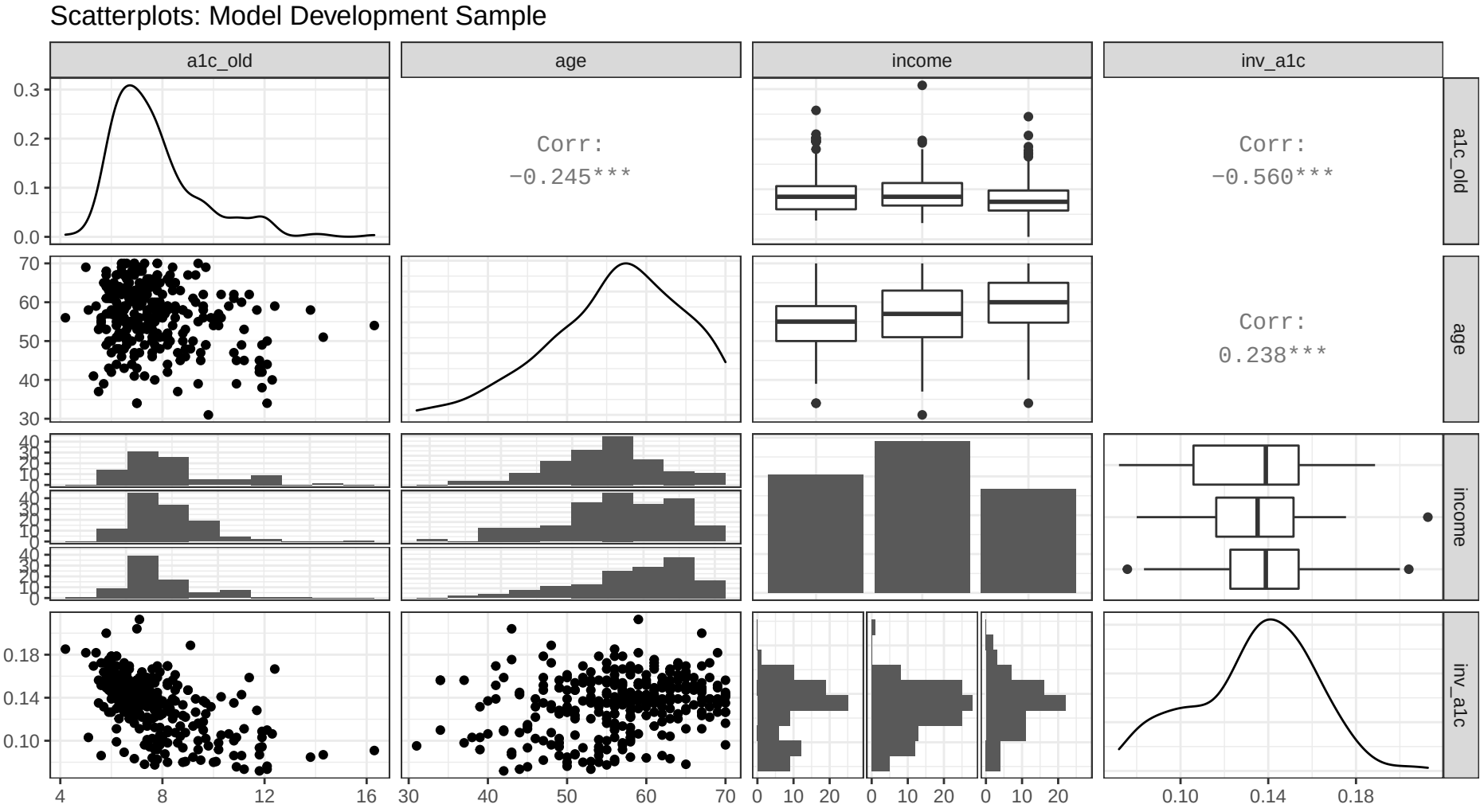
Inverse of Hemoglobin A1c

Model Development Sample: 288 adults with diabetes



This is where we stopped after Class 22. We'll restart here in Class 23.

Scatterplot Matrix (code on next slide)



Scatterplot Matrix (Code)

```
dm1_cc_train %>%
  mutate(inv_a1c = 1/a1c) %>%
  select(a1c_old, age, income, inv_a1c) %>%
  ggpairs(.,
    title = "Scatterplots: Model Development Sample",
    lower = list(combo = wrap("facethist", bins = 10)))
```

Note that `ggpairs` comes from the `GGally` package.

- If you have more than 4-5 predictors, it's usually necessary to split this up into two or more scatterplot matrices, each of which should include the outcome.
- I'd always put the outcome last in my selection here. That way, the bottom row will show the most important scatterplots, with the outcome on the Y axis, and each predictor, in turn on the X.

Three Regression Models We'll Fit

- Remember we're using the model development sample here.
- Let's work with the (1/a1c) transformation.

```
mod_1 <- lm((1/a1c) ~ a1c_old, data = dm1_cc_train)

mod_2 <- lm((1/a1c) ~ a1c_old + age, data = dm1_cc_train)

mod_3 <- lm((1/a1c) ~ a1c_old + age + income,
            data = dm1_cc_train)
```

Assess the quality of fit for candidate models within the development sample.

Tidied coefficients (mod_1)

```
tidy_m1 <- tidy(mod_1, conf.int = TRUE, conf.level = 0.95)

tidy_m1 %>%
  select(term, estimate, std.error, p.value,
         conf.low, conf.high) %>%
  knitr::kable(digits = 4)
```

term	estimate	std.error	p.value	conf.low	conf.high
(Intercept)	0.2006	6e-03	0	0.1887	0.2124
a1c_old	-0.0087	8e-04	0	-0.0102	-0.0072

The Regression Equation (mod_1)

Use the `equatiomatic` package to help here. Note the use of **results = 'asis'** in the code chunk name.

```
extract_eq(mod_1, use_coefs = TRUE, coef_digits = 4,
           ital_vars = TRUE)
```

$$(1/a1c) = 0.2006 - 0.0087(a1c_old) + \epsilon$$

Summary of Fit Quality (mod_1)

```
glance(mod_1) %>%
  mutate(name = "mod_1") %>%
  select(name, r.squared, adj.r.squared,
         sigma, AIC, BIC) %>%
  knitr::kable(digits = c(0, 3, 3, 3, 0, 0))
```

name	r.squared	adj.r.squared	sigma	AIC	BIC
mod_1	0.313	0.311	0.023	-1356	-1345

Tidied coefficients (mod_2)

```
tidy_m2 <- tidy(mod_2, conf.int = TRUE, conf.level = 0.95)

tidy_m2 %>%
  select(term, estimate, std.error, p.value,
         conf.low, conf.high) %>%
  knitr::kable(digits = 4)
```

term	estimate	std.error	p.value	conf.low	conf.high
(Intercept)	0.1771	0.0124	0.0000	0.1526	0.2016
a1c_old	-0.0083	0.0008	0.0000	-0.0098	-0.0068
age	0.0004	0.0002	0.0325	0.0000	0.0007

The Regression Equation (mod_2)

Again, we'll use the `equatiomatic` package, with **results = 'asis'**.

```
extract_eq(mod_2, use_coefs = TRUE, coef_digits = 4,
           ital_vars = TRUE)
```

$$(1/a1c) = 0.1771 - 0.0083(a1c_old) + 4e - 04(age) + \epsilon$$

Summary of Fit Quality (mod_2)

```
glance(mod_2) %>%
  mutate(name = "mod_2") %>%
  select(name, r.squared, adj.r.squared,
         sigma, AIC, BIC) %>%
  knitr::kable(digits = c(0, 3, 3, 3, 0, 0))
```

name	r.squared	adj.r.squared	sigma	AIC	BIC
mod_2	0.324	0.319	0.023	-1358	-1344

Tidied coefficients (mod_3)

```
tidy_m3 <- tidy(mod_3, conf.int = TRUE, conf.level = 0.95)

tidy_m3 %>%
  select(term, estimate, se = std.error,
         low = conf.low, high = conf.high, p = p.value) %>%
  knitr::kable(digits = c(4,4,4,4,3))
```

term	estimate	se	low	high	p
(Intercept)	0.1771	0.0125	0.1524	0.202	0.0000
a1c_old	-0.0083	0.0008	-0.0098	-0.007	0.0000
age	0.0004	0.0002	0.0000	0.001	0.0404
incomeBetween_30-50K	0.0003	0.0032	-0.0060	0.007	0.9341
incomeHigher_than_50K	0.0011	0.0036	-0.0059	0.008	0.7522

The Regression Equation (mod_3)

Again, we'll use the equatiomatic package.

```
extract_eq(mod_3, use_coefs = TRUE, coef_digits = 4,
           ital_vars = TRUE, wrap = TRUE, terms_per_line = 2)
```

$$
\begin{aligned}
(1/a1c) = 0.1771 - 0.0083(a1c_old) + \\
4e-04(age) + 3e-04(income_{Between_30-50K}) + \\
0.0011(income_{Higher_than_50K}) + \epsilon
\end{aligned}
$$

Summary of Fit Quality (mod_3)

```
glance(mod_3) %>%
  mutate(name = "mod_3") %>%
  select(name, r.squared, adj.r.squared,
         sigma, AIC, BIC) %>%
  knitr::kable(digits = c(0, 3, 3, 3, 0, 0))
```

name	r.squared	adj.r.squared	sigma	AIC	BIC
mod_3	0.324	0.315	0.023	-1354	-1332

Could we have fit other predictor sets?

Perhaps an automated procedure, like stepwise regression, would suggest a better alternative?

- Three predictor candidates, so we could have used any of these predictor sets:
- `a1c_old` alone (our `mod_1`)
- `age` alone
- `income` alone
- `a1c_old` and `age` (our `mod_2`)
- `a1c_old` and `income`
- `age` and `income`
- `a1c_old`, `age` and `income` (our `mod_3`)

```
step(mod_3)
```

Stepwise Regression Results (part 1 of 2)

We'll try backwards elimination, where we let R's `step` function start with the full model (mod_3) including all three predictors, and then remove the predictor whose removal causes the largest drop in AIC, until we reach a point where eliminating another predictor will not improve the AIC.

- Remember the smaller (more negative, here) the AIC, the better.

```
step(mod_3)
```

```
Start:  AIC=-2173.59
(1/a1c) ~ a1c_old + age + income

          Df Sum of Sq     RSS     AIC
- income   2  0.000056 0.14682 -2177.5
<none>                 0.14676 -2173.6
- age      1  0.002198 0.14896 -2171.3
- a1c_old  1  0.057786 0.20455 -2080.0
```

Stepwise Regression Results (part 2 of 2)

```
Step:  AIC=-2177.47
(1/a1c) ~ a1c_old + age

          Df Sum of Sq     RSS     AIC
<none>                 0.14682 -2177.5
- age      1  0.002377 0.14920 -2174.8
- a1c_old  1  0.058050 0.20487 -2083.5

Call:
lm(formula = (1/a1c) ~ a1c_old + age, data = dm1_cc_train)

Coefficients:
(Intercept)      a1c_old          age
  0.1771207   -0.0082934    0.0003593
```

and we wind up here with just our `mod_2`.

An Important Point

- There is an **enormous** amount of evidence that variable selection causes severe problems in estimation and inference.
- Stepwise regression, in particular, is an egregiously bad choice, as we'll show in 432.
- Disappointingly, there really isn't a good choice. The task itself just isn't one we can do well.

More on this in 432.

Comparing Summary Measures of Fit

in the development (model training) sample...

```
bind_rows(glance(mod_1), glance(mod_2), glance(mod_3)) %>%
  mutate(model_vars = c("1_a1c_old", "2_+age", "3_+income")) %
  select(model_vars, r2 = r.squared, adj_r2 = adj.r.squared,
         sigma, AIC, BIC, df, df_res = df.residual) %>%
  kable(digits = c(0, 4, 4, 5, 1, 0, 0, 0))
```

model_vars	r2	adj_r2	sigma	AIC	BIC	df	df_res
1_a1c_old	0.3131	0.3107	0.02284	-1355.5	-1345	1	286
2_+age	0.3241	0.3193	0.02270	-1358.2	-1344	2	285
3_+income	0.3243	0.3148	0.02277	-1354.3	-1332	4	283

In the data we used to build the model, these are our results.

Which Model Looks Best In-Sample?

For each of these summaries, which model looks best in the training sample?

model	vars	r2	adj_r2	sigma	AIC	BIC
mod_1	a1c_old	0.3131	0.3107	0.02284	-1355.5	-1345
mod_2	+ age	0.3241	0.3193	0.02270	-1358.2	-1344
mod_3	+ income	0.3243	0.3148	0.02277	-1354.3	-1332

- By r^2, the largest model will always look best (raw r^2 is greedy)
- Adjusted r^2 penalizes for lack of parsimony. Model 2 looks best there.
- For σ, AIC and BIC, we want small (more negative) values.
  - Model 2 looks best by σ and AIC, as well.
  - Model 1 looks a little better than Model 2 by BIC.
- Overall, what should we conclude about in-sample fit quality?

Check adherence to regression assumptions in the development sample.

Using `augment` to add fits, residuals, etc.

```
aug1 <- augment(mod_1, data = dm1_cc_train) %>%
  mutate(inv_a1c = 1/a1c) # add in our model's outcome
```

`aug1` includes all variables in `dm_cc_train` to which we've added:

- `inv_a1c` = 1/`a1c`, the transformed outcome that `mod_1` predicts
- `.fitted` = fitted (predicted) values of 1/`a1c`
- `.resid` = residual (observed outcome - fitted outcome) values, so that larger values (positive or negative) mean poorer fit points
- `.std.resid` = standardized residuals (residuals scaled to SD = 1, remember that the residual mean is already 0)
- `.hat` statistic = measures *leverage* (larger values of `.hat` indicate unusual combinations of predictor values)
- `.cooksd` = Cook's distance (or Cook's d), a measure of the subject's *influence* on the model (larger Cook's d values indicate that removing the point will materially change the model's coefficients)
- plus `.sigma` = estimated σ if this point is dropped from the model

augment results for the first 2 subjects

```
aug1 %>% select(subject, a1c:income, inv_a1c) %>%
  tail(2) %>% kable(dig = 3)
```

subject	a1c	a1c_old	age	income	inv_a1c
S-135	6.6	8.6	47	Below_30K	0.152
S-125	9.9	7.8	58	Between_30-50K	0.101

```
aug1 %>% select(subject, .fitted:.cooksd) %>%
  tail(2) %>% kable(dig = 3)
```

subject	.fitted	.resid	.std.resid	.hat	.sigma	.cooksd
S-135	0.126	0.026	1.133	0.004	0.023	0.003
S-125	0.133	-0.032	-1.388	0.003	0.023	0.003

augment for models mod_2 and mod_3

We need the augment results for our other two models: mod_2 and mod_3.

```
aug2 <- augment(mod_2, data = dm1_cc_train) %>%
  mutate(inv_a1c = 1/a1c) # add in our model's outcome
```

```
aug3 <- augment(mod_3, data = dm1_cc_train) %>%
  mutate(inv_a1c = 1/a1c) # add in our model's outcome
```

Checking Regression Assumptions

Four key assumptions we need to think about:

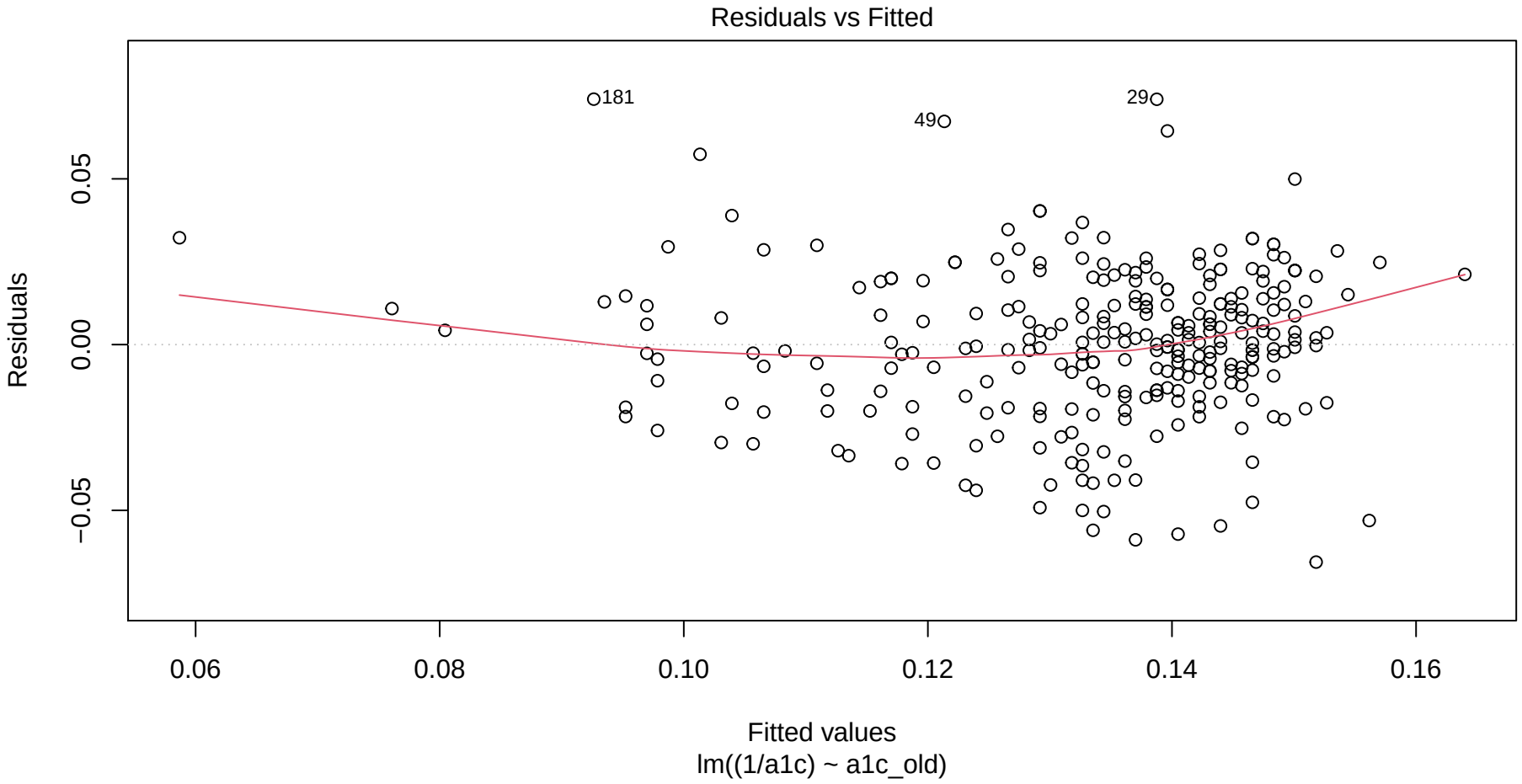
1. Linearity
2. Constant Variance (Homoscedasticity)
3. Normality
4. Independence

How do we assess 1, 2, and 3? Residual plots.

There are five automated ones that we could obtain using `plot(mod_1)`...

Residuals vs. Fitted Values Plot (Model mod_1)

```
plot(mod_1, which = 1)
```



Which points are highlighted in that plot?

Note that the points labeled 181, 49 and 29 are the 181st, 49th and 29th rows in our `dm1_cc_train` data file, or, equivalently, in our `aug1` file.

```
aug1 %>% slice(c(181, 49, 29)) %>% select(a1c:.resid)
```

```
# A tibble: 3 x 7
    a1c a1c_old   age income     subject .fitted .resid
  <dbl>   <dbl> <int> <fct>      <chr>     <dbl>  <dbl>
1   6      12.4    59 Below_30K  S-105    0.0926 0.0740
2   5.3     9.1    48 Below_30K  S-071    0.121  0.0673
3   4.7     7.1    59 Between_3~ S-164    0.139  0.0740
```

These are subjects `S-105`, `S-071`, `S-164`.

Another way to confirm who the plot is identifying

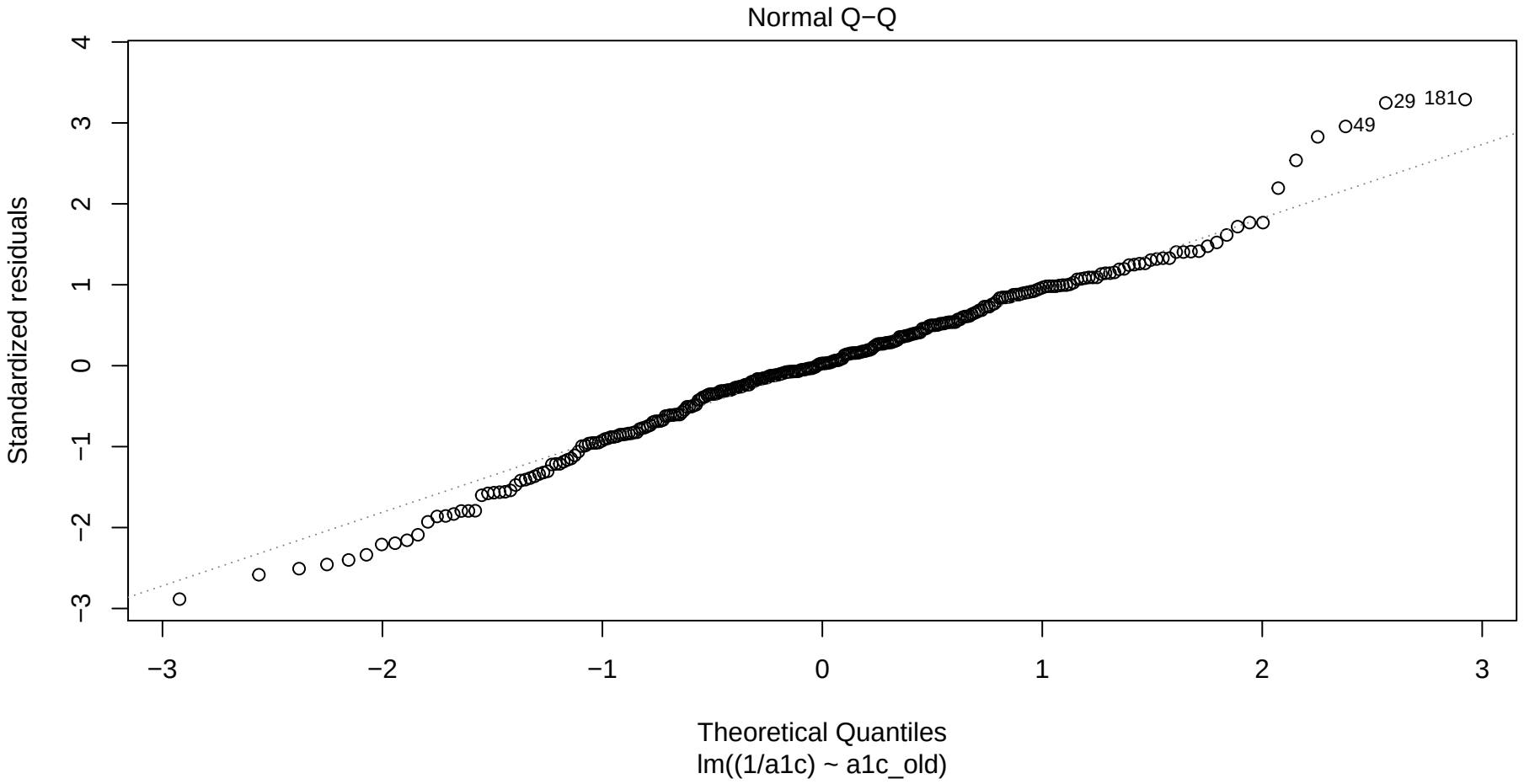
As mentioned, we think the identifiers (181, 49 and 129) of the points with the largest residual (in absolute value) describe subjects S-105, S-071 and S-164, respectively. Does this make sense?

```
aug1 %>% select(subject, .resid) %>%
  arrange(desc(abs(.resid))) %>% head()
```

```
# A tibble: 6 x 2
  subject  .resid
  <chr>     <dbl>
1 S-105    0.0740
2 S-164    0.0740
3 S-071    0.0673
4 S-168   -0.0656
5 S-052    0.0645
6 S-214   -0.0589
```

Normal Q-Q of Standardized Residuals (mod_1)

```
plot(mod_1, which = 2)
```



Are the outliers we see there completely out of line?

```r
nrow(aug1)
```

```
[1] 288
```

```r
aug1 %>% select(subject, .std.resid) %>%
  arrange(desc(abs(.std.resid)))
```

```
# A tibble: 288 x 2
   subject .std.resid
   <chr>        <dbl>
 1 S-105         3.29
 2 S-164         3.25
 3 S-071         2.96
 4 S-168        -2.88
 5 S-052         2.83
 6 S-214        -2.58
 7 S-001         2.54
 8 S-258        -2.51
```

Is a Z score of 3.29 for the biggest outlier scary here?

```r
outlierTest(mod_1)
```

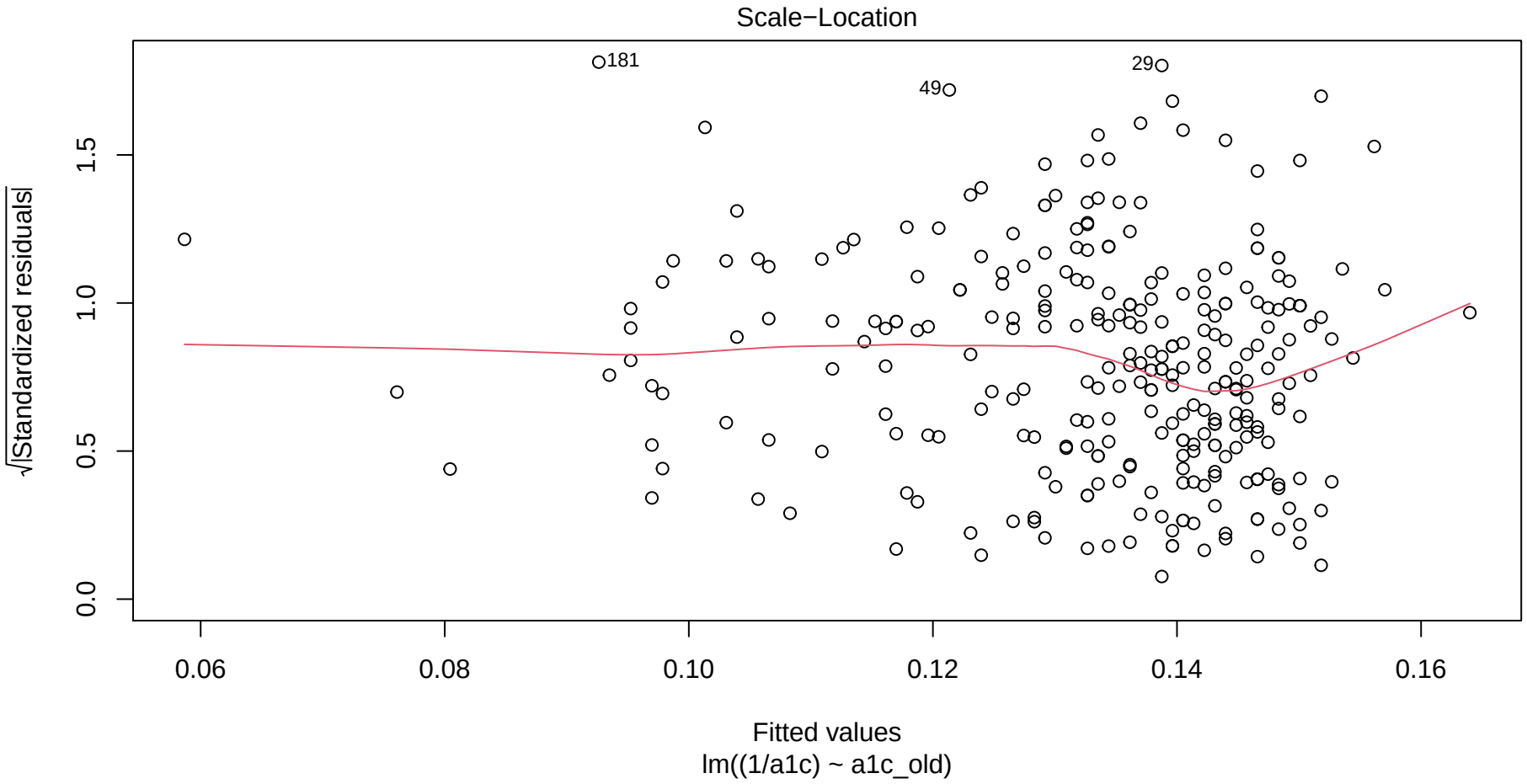
```
No Studentized residuals with Bonferroni p < 0.05
Largest |rstudent|:
    rstudent unadjusted p-value Bonferroni p
181 3.346538         0.00092815      0.26731
```

For now, a studentized residual is just another way to standardize the residuals that has some useful properties in this setting.

- There's no indication that having a maximum absolute value of 3.34 in a sample of 288 studentized residuals is a major concern about the assumption of Normality, given the Bonferroni p value of 0.27.

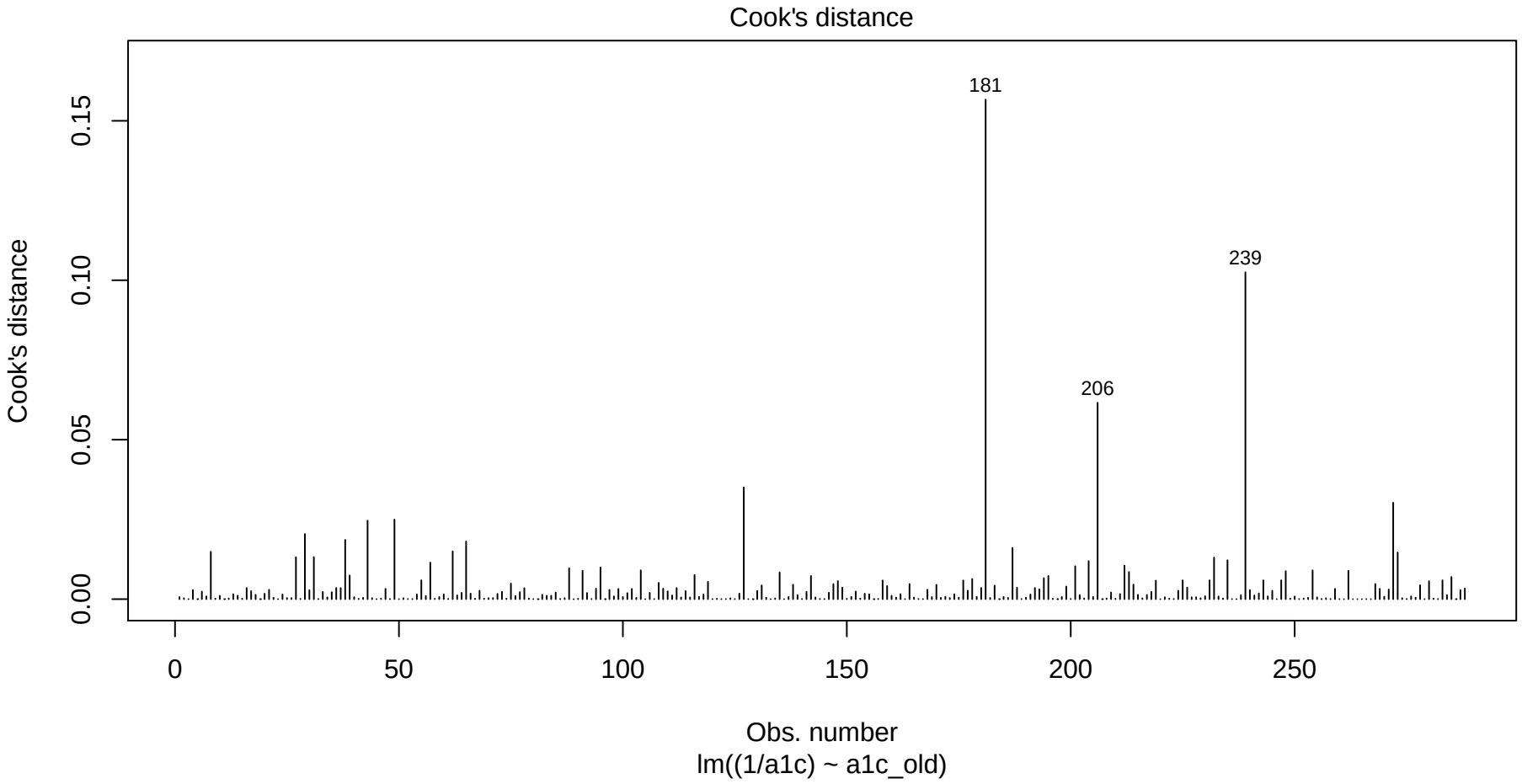
Scale-Location: Check for heteroscedasticity (mod_1)

```
plot(mod_1, which = 3)
```



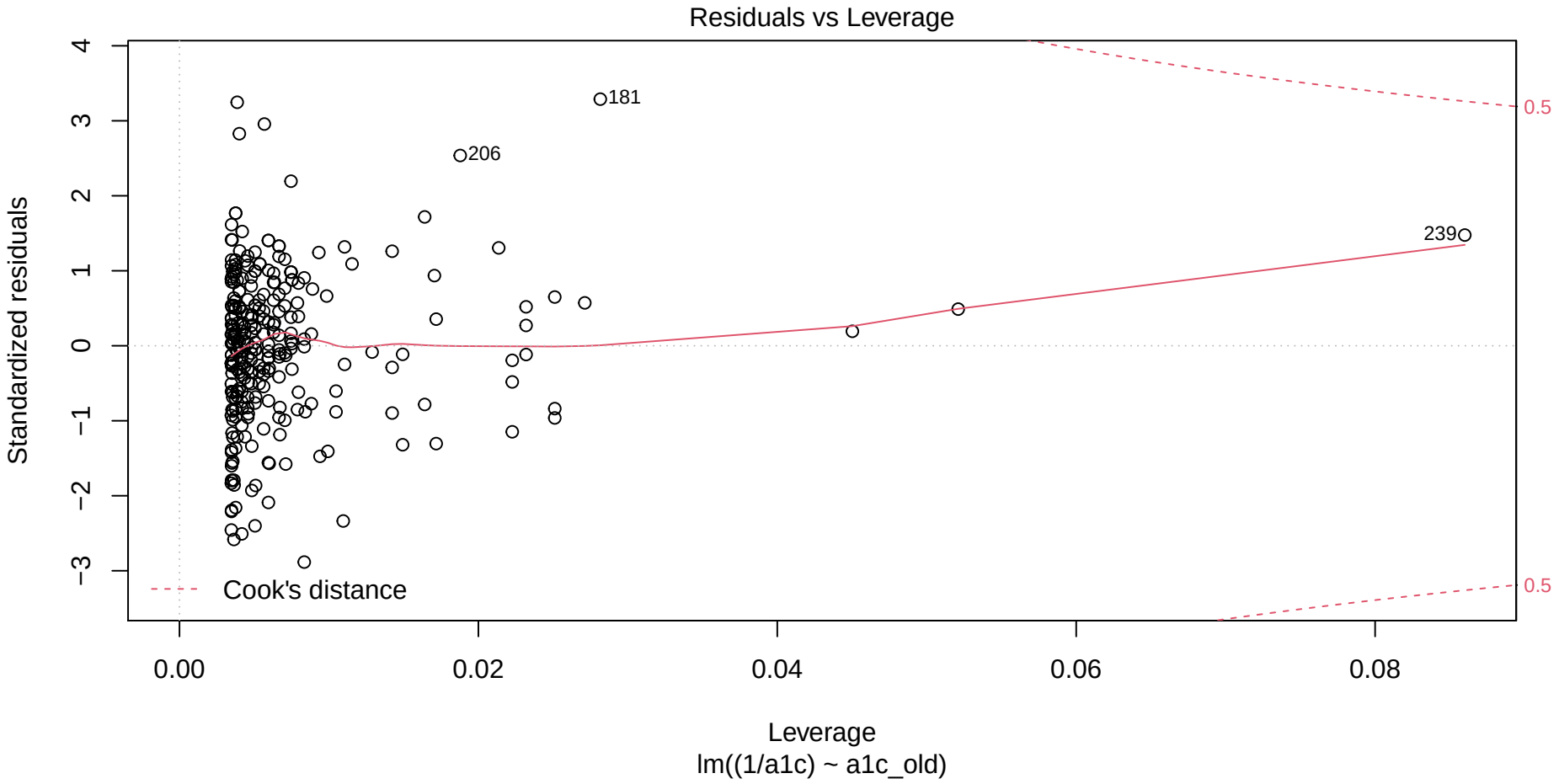
Index plot of Cook's distance for influence (mod_1)

```
plot(mod_1, which = 4)
```



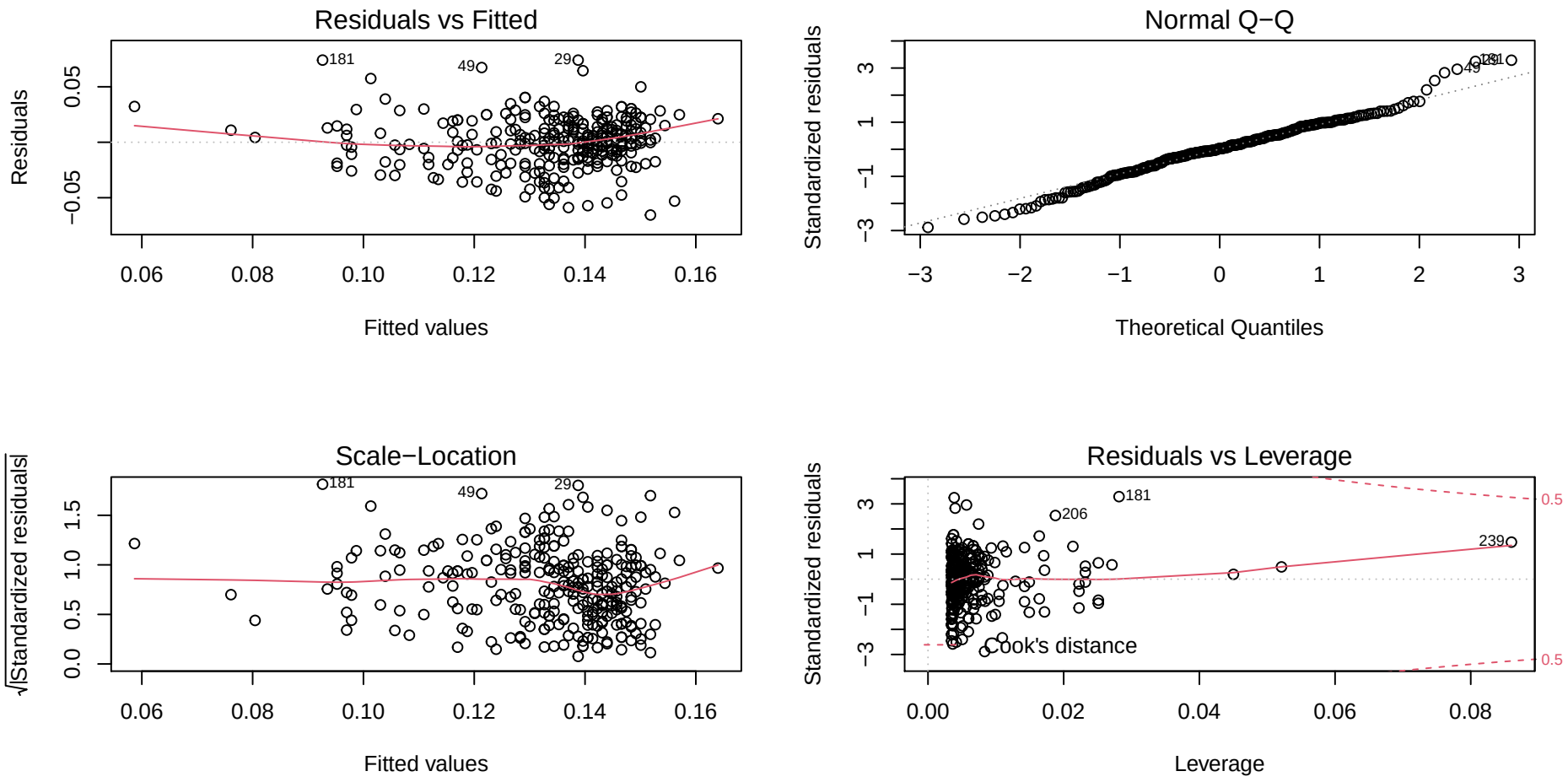
Residuals, Leverage and Influence plot (mod_1)

```
plot(mod_1, which = 5)
```

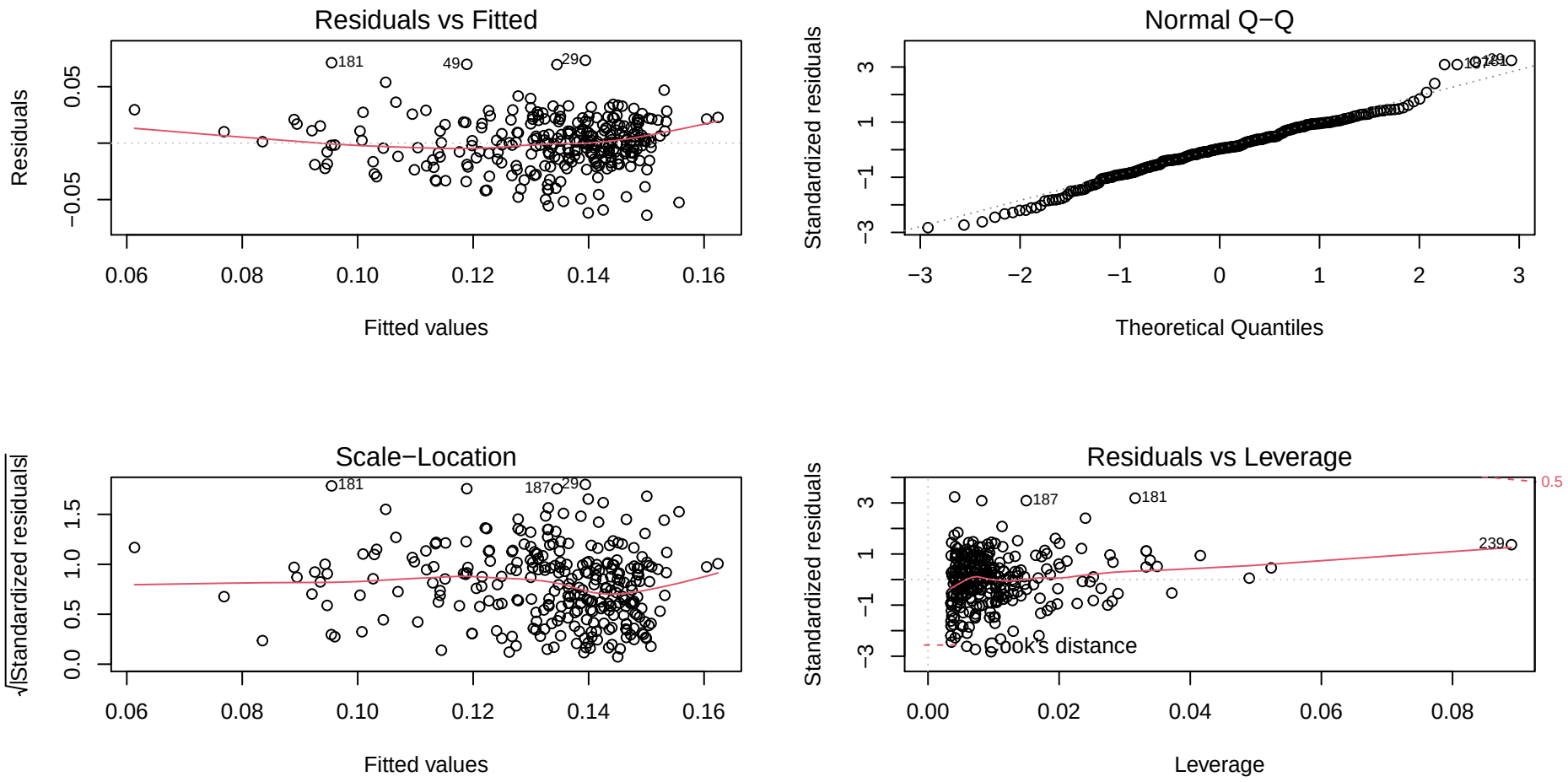


Residual Plots for Model mod_1

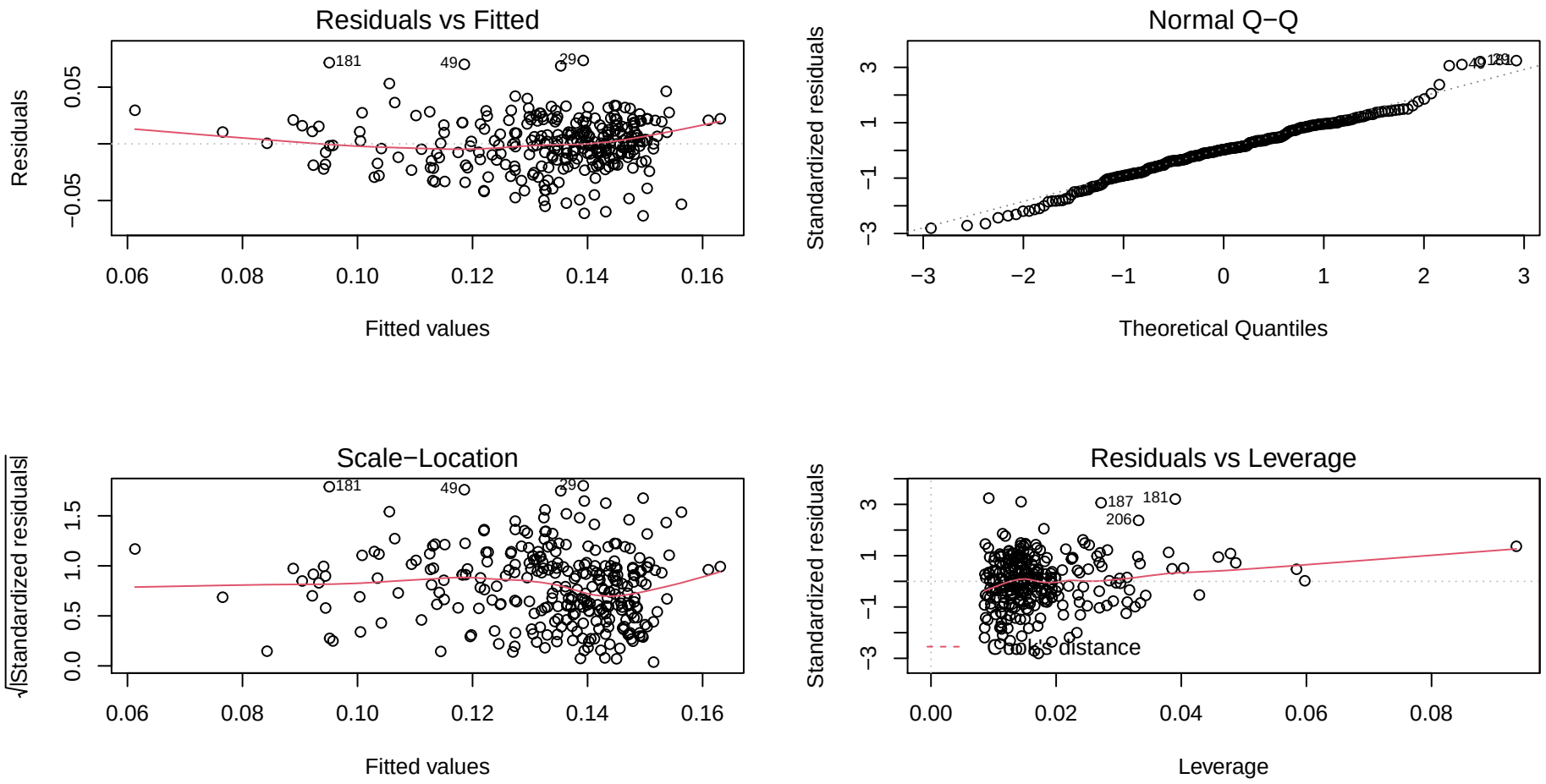
```
par(mfrow = c(2,2)); plot(mod_1); par(mfrow = c(1,1))
```



Residual Plots for Model `mod_2`



Residual Plots for Model `mod_3`



Is collinearity a serious issue here?

```
car::vif(mod_3)
```

```
            GVIF Df GVIF^(1/(2*Df))
a1c_old 1.065707  1        1.032331
age     1.093345  1        1.045631
income  1.035204  2        1.008687
```

- Collinearity = correlated predictors
- (generalized) Variance Inflation Factor tells us something about how the standard errors of our coefficients are inflated as a result of correlation between predictors.
  - We tend to worry most about VIFs in this output that exceed 5.
  - Remember that the scatterplot matrix didn't suggest any strong correlations between our predictors.

What would we do if we had strong collinearity? Drop a predictor?

Conclusions so far?

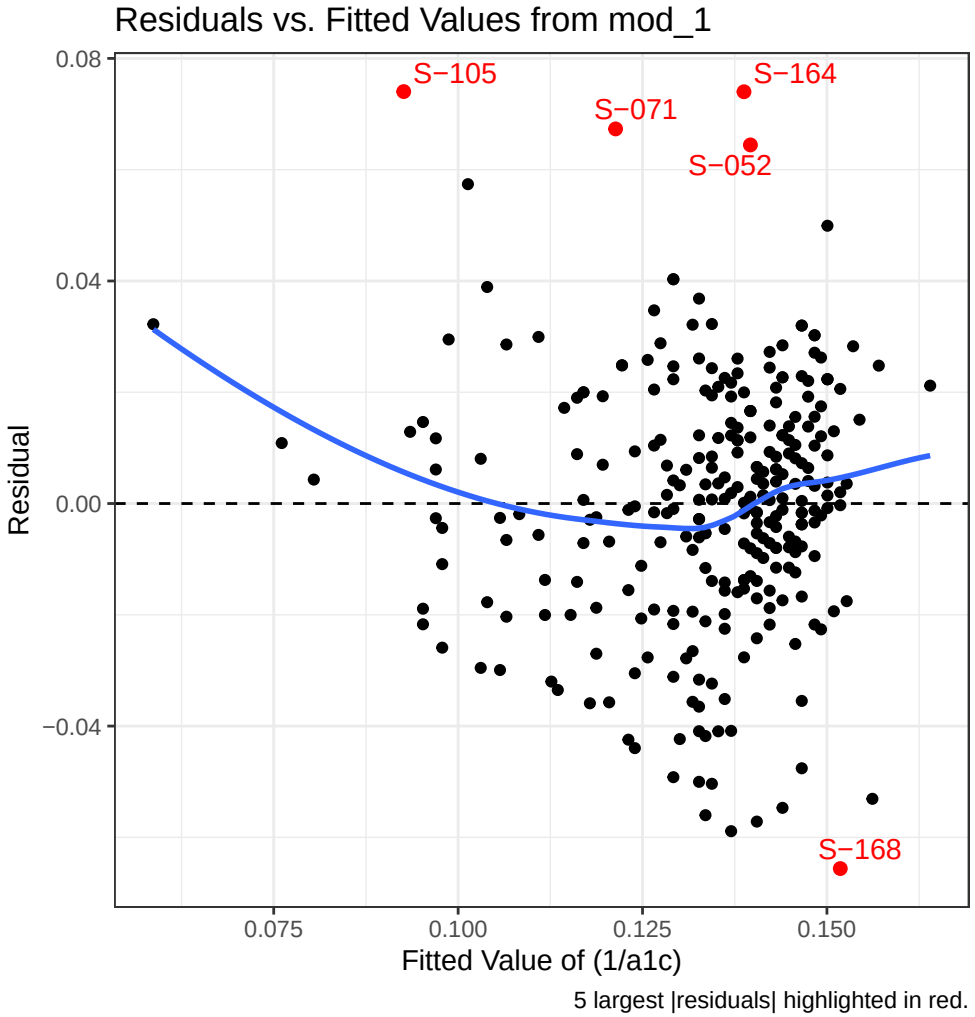
1. In-sample model predictions are about equally accurate for each of the three models. Model 2 looks better in terms of adjusted R^2 and AIC, but model 1 looks better on BIC. There's really not much to choose from there.
2. Residual plots look similarly reasonable for linearity, Normality and constant variance in all three models.

Using `ggplot2` to create these residual plots?

1. Residuals vs. Fitted Values plots are straightforward, with the use of the `augment` function from the `broom` package.
   - We can also plot residuals against individual predictors, if we like.
2. Similarly, plots to assess the Normality of the residuals, like a Normal Q-Q plot, are straightforward, and can use either raw residuals or standardized residuals.
3. The scale-location plot of the square root of the standardized residuals vs. the fitted values is also pretty straightforward.
4. The `augment` function can be used to obtain Cook's distance, standardized residuals and leverage values, so we can mimic both the index plot (of Cook's distance) as well as the residuals vs. leverage plot with Cook's distance contours, if we like.

Demonstrations on the next few slides, followed by the code.

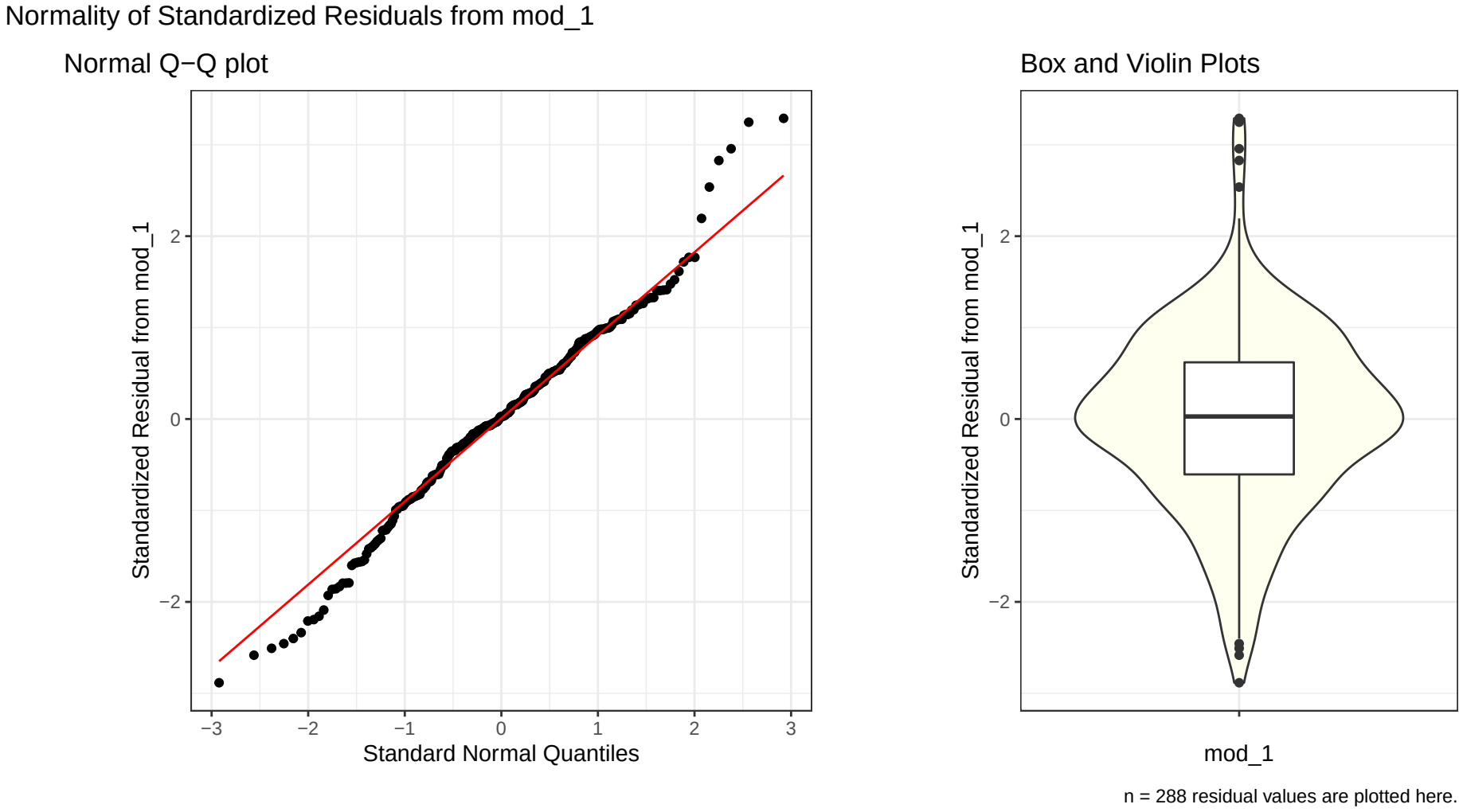
Residuals vs. Fitted Values Plot via ggplot2



Code for Residuals vs. Fitted Values

```
ggplot(aug1, aes(x = .fitted, y = .resid)) +
  geom_point() +
  geom_point(data = aug1 %>%
               slice_max(abs(.resid), n = 5),
             col = "red", size = 2) +
  geom_text_repel(data = aug1 %>%
               slice_max(abs(.resid), n = 5),
               aes(label = subject), col = "red") +
  geom_abline(intercept = 0, slope = 0, lty = "dashed") +
  geom_smooth(method = "loess", formula = y ~ x, se = F) +
  labs(title = "Residuals vs. Fitted Values from mod_1",
       caption = "5 largest |residuals| highlighted in red.",
       x = "Fitted Value of (1/a1c)", y = "Residual") +
  theme(aspect.ratio = 1)
```

Normality of Standardized Residuals via ggplot2



Code for Normality Checks (1 of 2)

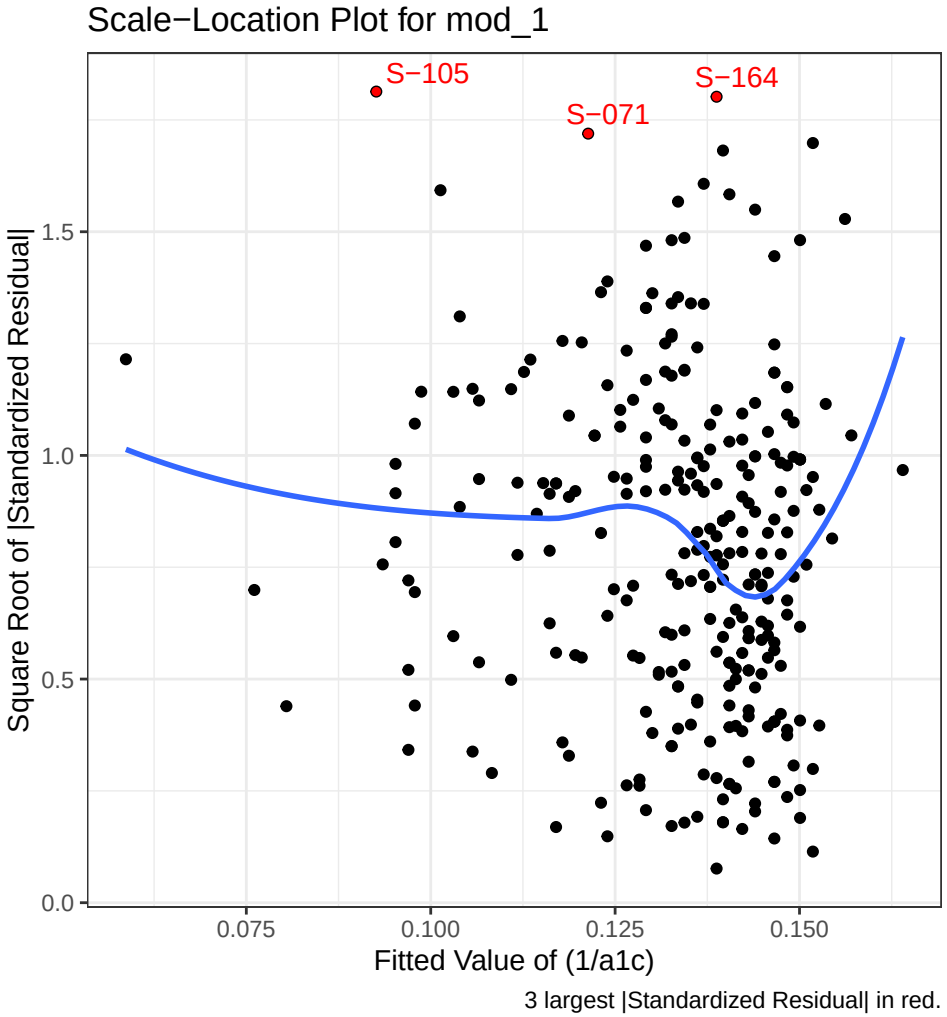
```
p1 <- ggplot(aug1, aes(sample = .std.resid)) +
  geom_qq() +
  geom_qq_line(col = "red") +
  labs(title = "Normal Q-Q plot",
       y = "Standardized Residual from mod_1",
       x = "Standard Normal Quantiles") +
  theme(aspect.ratio = 1)

p2 <- ggplot(aug1, aes(y = .std.resid, x = "")) +
  geom_violin(fill = "ivory") +
  geom_boxplot(width = 0.3) +
  labs(title = "Box and Violin Plots",
       y = "Standardized Residual from mod_1",
       x = "mod_1")
```

. . . continues on next slide

Code for Normality Checks (2 of 2)

```
p1 + p2 +
  plot_layout(widths = c(2, 1)) +
  plot_annotation(
    title = "Normality of Standardized Residuals from mod_1",
    caption = paste0("n = ",
                     nrow(aug1 %>% select(.std.resid)),
                     " residual values are plotted here."))
```

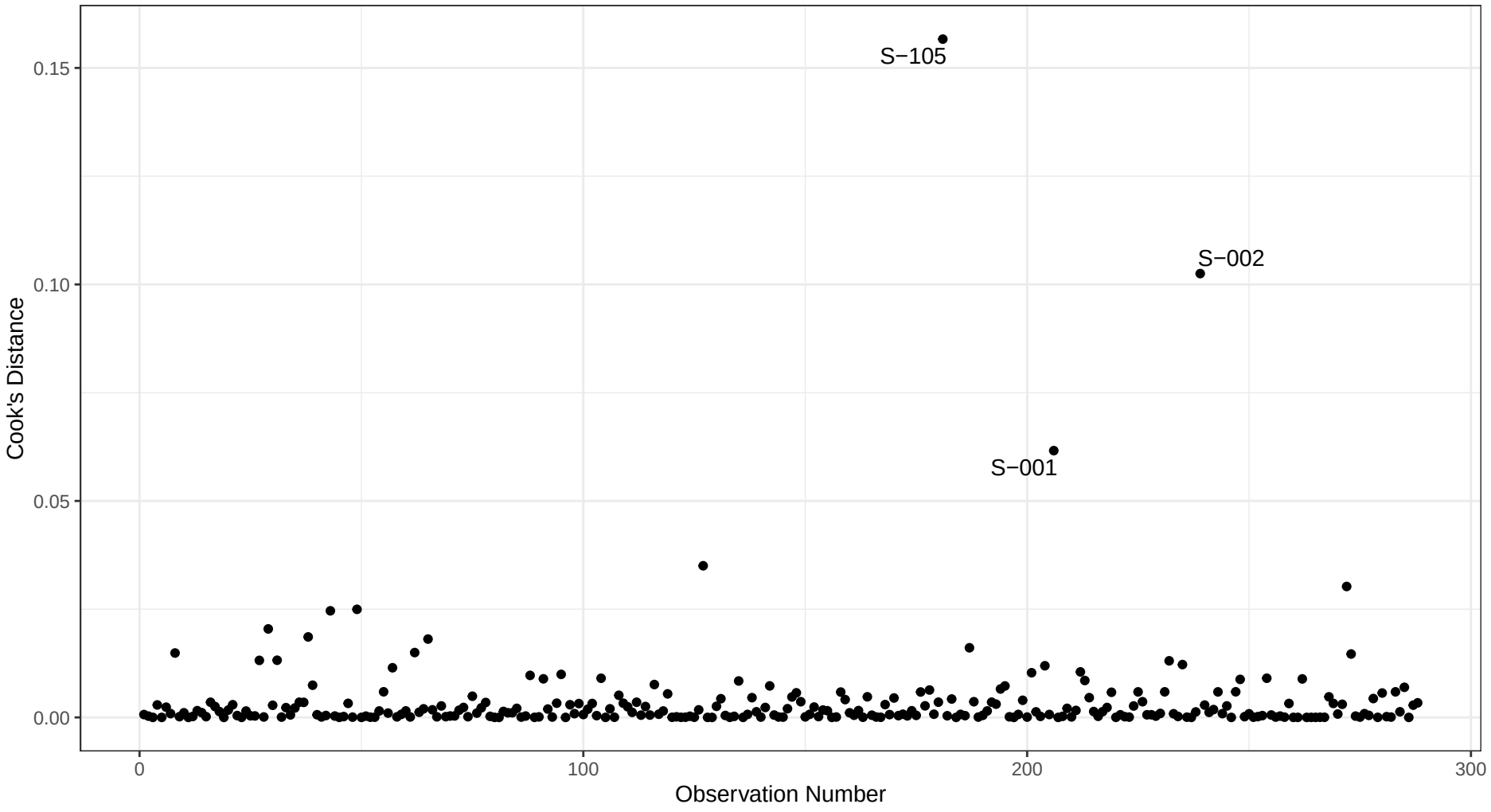
Scale-Location Plot via ggplot2



Code for Scale-Location Plot

```
ggplot(aug1, aes(x = .fitted, y = sqrt(abs(.std.resid)))) +
  geom_point() +
  geom_point(data = aug1 %>%
               slice_max(sqrt(abs(.std.resid)), n = 3),
             col = "red", size = 1) +
  geom_text_repel(data = aug1 %>%
               slice_max(sqrt(abs(.std.resid)), n = 3),
               aes(label = subject), col = "red") +
  geom_smooth(method = "loess", formula = y ~ x, se = F) +
  labs(title = "Scale-Location Plot for mod_1",
       caption = "3 largest |Standardized Residual| in red.",
       x = "Fitted Value of (1/a1c)",
       y = "Square Root of |Standardized Residual|") +
  theme(aspect.ratio = 1)
```

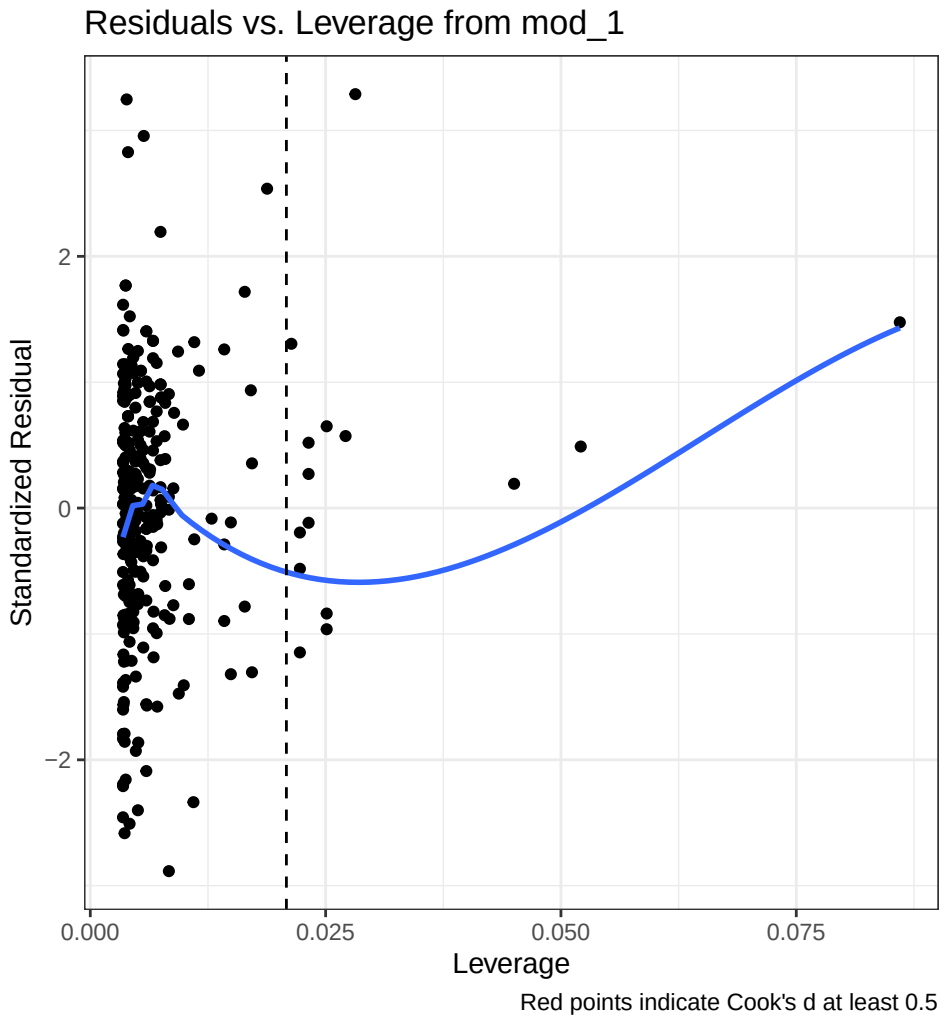
Cook's Distance Index Plot via `ggplot2`



Code for Cook's Distance Index Plot

```
aug1_extra <- aug1 %>%
  mutate(obsnum = 1:nrow(aug1 %>% select(.cooksd)))

ggplot(aug1_extra, aes(x = obsnum, y = .cooksd)) +
  geom_point() +
  geom_text_repel(data = aug1_extra %>%
                  slice_max(.cooksd, n = 3),
                  aes(label = subject)) +
  labs(x = "Observation Number",
       y = "Cook's Distance")
```

Residuals vs. Leverage Plot via ggplot2



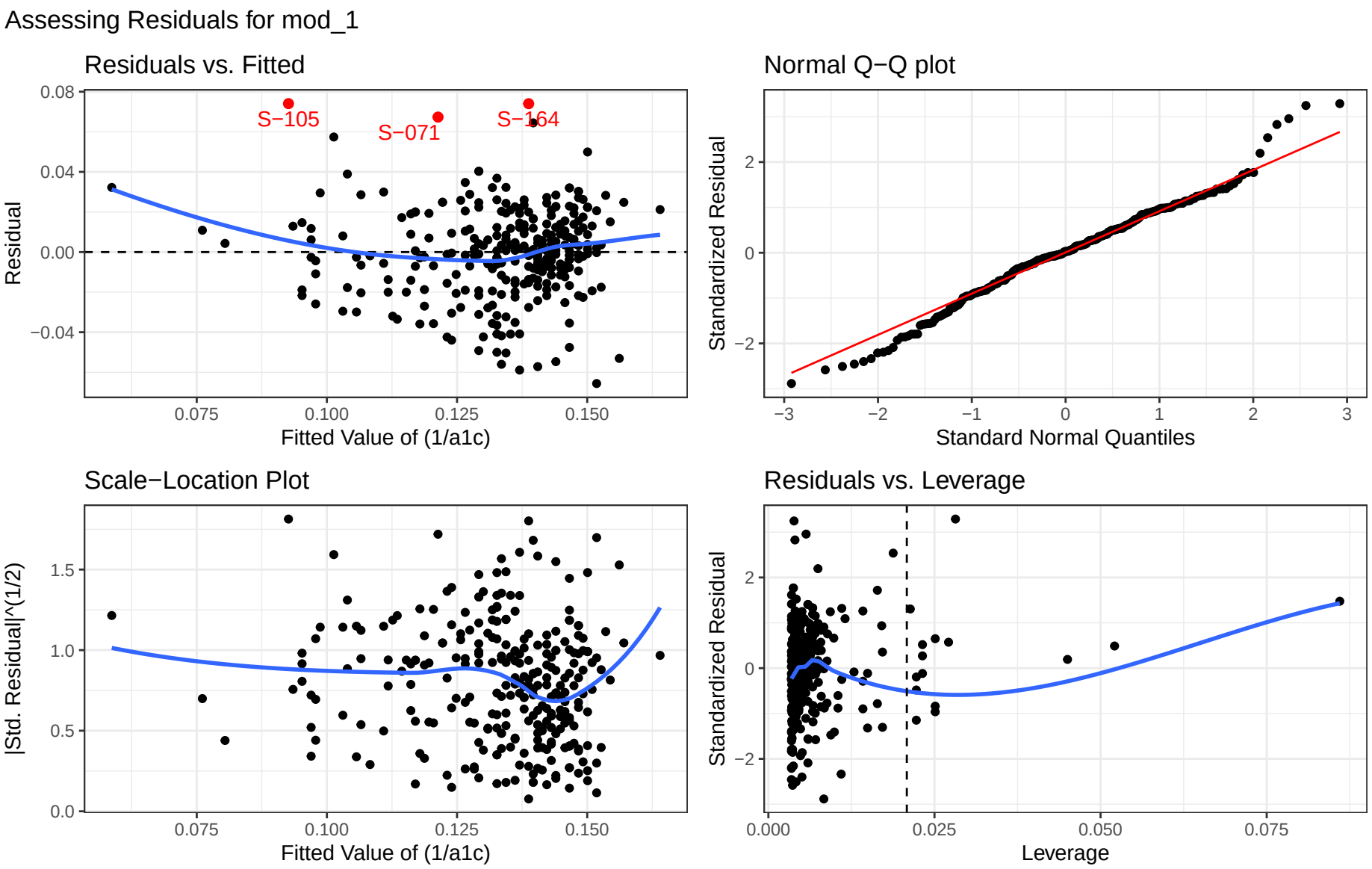
- Points with Cook's d >= 0.5 would be highlighted and in red.
- Points right of the dashed line have high leverage, by one standard.

Code for Residuals vs. Leverage Plot

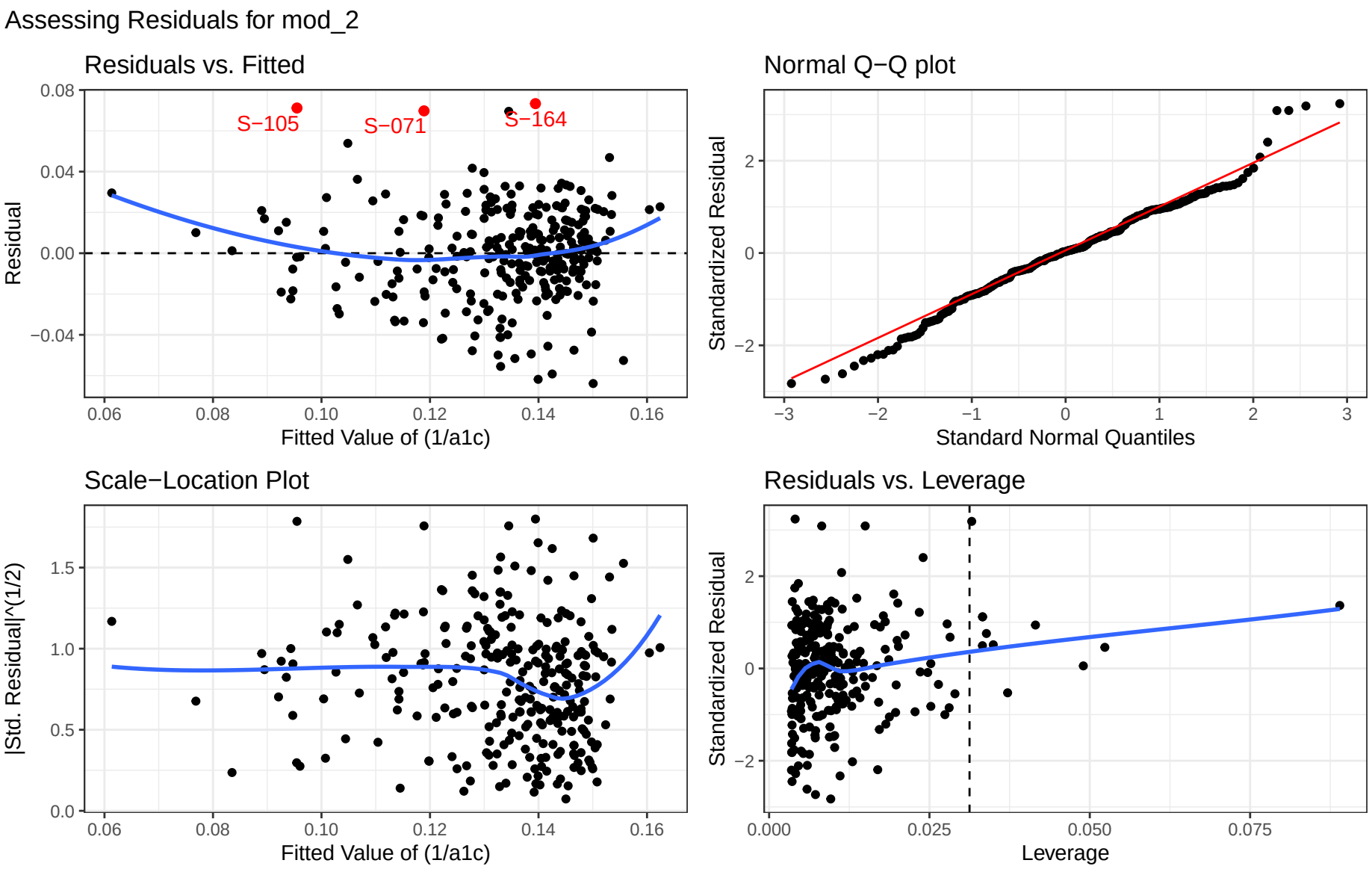
```
ggplot(aug1, aes(x = .hat, y = .std.resid)) +
  geom_point() +
  geom_point(data = aug1 %>% filter(.cooksd >= 0.5),
             col = "red", size = 2) +
  geom_text_repel(data = aug1 %>% filter(.cooksd >= 0.5),
                  aes(label = subject), col = "red") +
  geom_smooth(method = "loess", formula = y ~ x, se = F) +
  geom_vline(aes(xintercept = 3*mean(.hat)), lty = "dashed") +
  labs(title = "Residuals vs. Leverage from mod_1",
       caption = "Red points indicate Cook's d at least 0.5",
       x = "Leverage", y = "Standardized Residual") +
  theme(aspect.ratio = 1)
```

- Points with more than 3 times the average leverage are identified as highly leveraged by some people, hence my dashed vertical line.

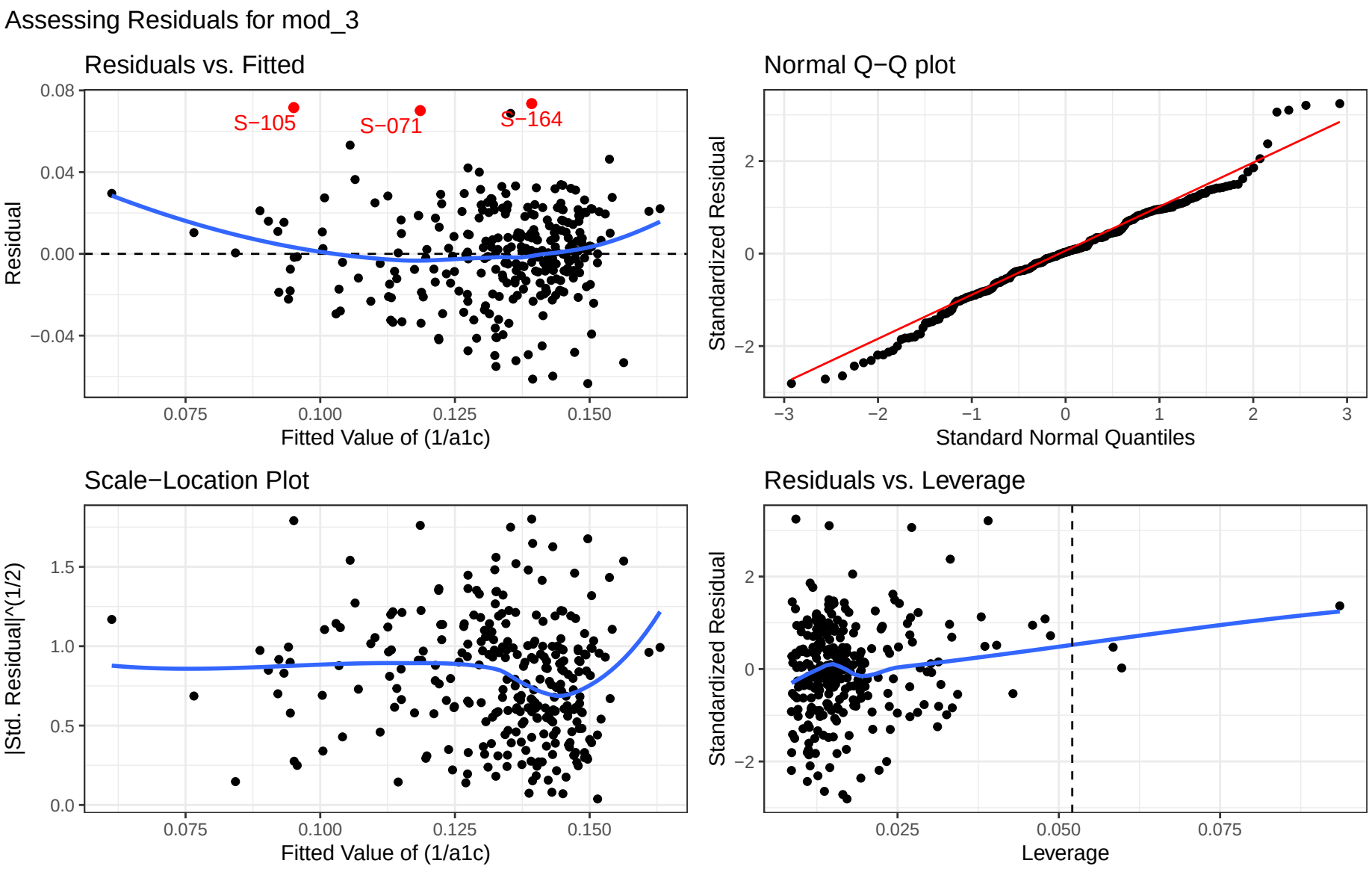
Main 4 Residual Plots for `mod_1` (via `ggplot2`)



Main 4 Residual Plots for `mod_2` (via `ggplot2`)



Main 4 Residual Plots for `mod_3` (via `ggplot2`)



Conclusions so far? (repeating what we said earlier)

1. In-sample model predictions are about equally accurate for each of the three models. Model 2 looks better in terms of adjusted R^2 and AIC, but model 1 looks better on BIC. There's really not much to choose from there.
2. Residual plots look fairly reasonable for linearity, Normality and constant variance in all three models.

When you have candidates, assess them based on the accuracy of the predictions they make for the data held out (and thus not used in building the models.)

Calculate prediction errors for mod_1 in test sample

The augment function in the broom package will create predictions within our new sample, but we want to back-transform these predictions so that they are on the original scale (a1c, rather than our transformed regression outcome 1/a1c). Since the way to back out of the inverse transformation is to take the inverse again, we will take the inverse of the fitted values provided by augment and then calculate residuals on the original scale, as follows. . .

```
test_m1 <- augment(mod_1, newdata = dm1_cc_test) %>%
  mutate(name = "mod_1", fit_a1c = 1 / .fitted,
         res_a1c = a1c - fit_a1c)
```

What does `test_m1` now include?

```
test_m1 %>%
  select(subject, a1c, fit_a1c, res_a1c, a1c_old,
         age, income) %>%
  head() %>%
  knitr::kable(digits = c(0, 1, 2, 2, 1, 0, 0))
```

subject	a1c	fit_a1c	res_a1c	a1c_old	age	income
S-010	6.7	7.07	-0.37	6.8	63	Below_30K
S-011	16.7	13.61	3.09	14.6	54	Below_30K
S-012	12.2	9.78	2.42	11.3	52	Below_30K
S-014	8.4	7.96	0.44	8.6	44	Between_30-50K
S-015	5.7	6.63	-0.93	5.7	52	Between_30-50K
S-017	6.6	6.86	-0.26	6.3	58	Between_30-50K

Gather test-sample prediction errors for models 2, 3

```
test_m2 <- augment(mod_2, newdata = dm1_cc_test) %>%
  mutate(name = "mod_2", fit_a1c = 1 / .fitted,
         res_a1c = a1c - fit_a1c)

test_m3 <- augment(mod_3, newdata = dm1_cc_test) %>%
  mutate(name = "mod_3", fit_a1c = 1 / .fitted,
         res_a1c = a1c - fit_a1c)
```

Combine test sample results from the three models

```
test_comp <- bind_rows(test_m1, test_m2, test_m3) %>%
  arrange(subject, name)

test_comp %>% select(name, subject, a1c, fit_a1c, res_a1c,
                     a1c_old, age, income) %>%
  slice(1:3, 7:9) %>%
  knitr::kable(digits = c(0, 0, 1, 2, 2, 1, 0, 0))
```

name	subject	a1c	fit_a1c	res_a1c	a1c_old	age	income
mod_1	S-010	6.7	7.07	-0.37	6.8	63	Below_30K
mod_2	S-010	6.7	6.98	-0.28	6.8	63	Below_30K
mod_3	S-010	6.7	7.00	-0.30	6.8	63	Below_30K
mod_1	S-012	12.2	9.78	2.42	11.3	52	Below_30K
mod_2	S-012	12.2	9.80	2.40	11.3	52	Below_30K
mod_3	S-012	12.2	9.83	2.37	11.3	52	Below_30K

What do we do to compare the test-sample errors?

Given this tibble, including predictions and residuals from the three models on our test data, we can now:

1. Visualize the prediction errors from each model.
2. Summarize those errors across each model.
3. Identify the "worst fitting" subject for each model in the test sample.

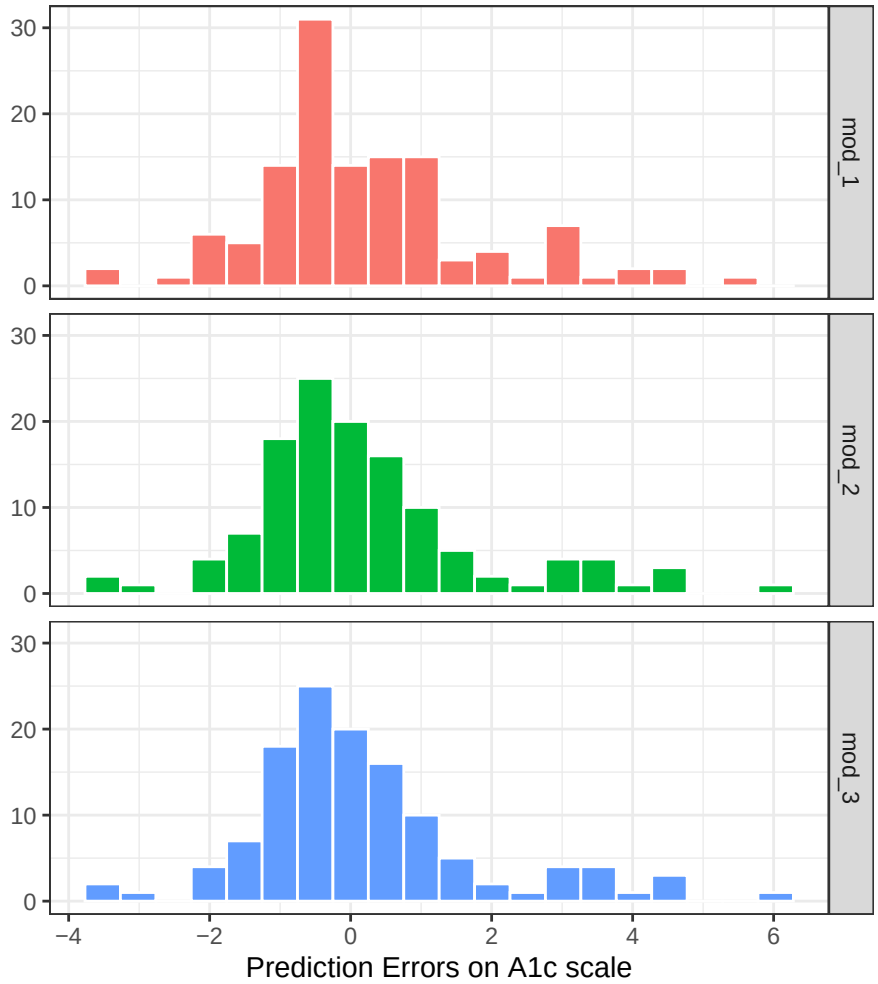
Visualize the prediction errors

```
ggplot(test_comp, aes(x = res_a1c, fill = name)) +
  geom_histogram(bins = 20, col = "white") +
  facet_grid (name ~ .) + guides(fill = FALSE)
```

or maybe

```
ggplot(test_comp, aes(x = name, y = res_a1c, fill = name)) +
  geom_violin(alpha = 0.3) +
  geom_boxplot(width = 0.3, outlier.shape = NA) +
  geom_jitter(height = 0, width = 0.1) +
  guides(fill = FALSE)
```

Test-Sample Prediction Errors



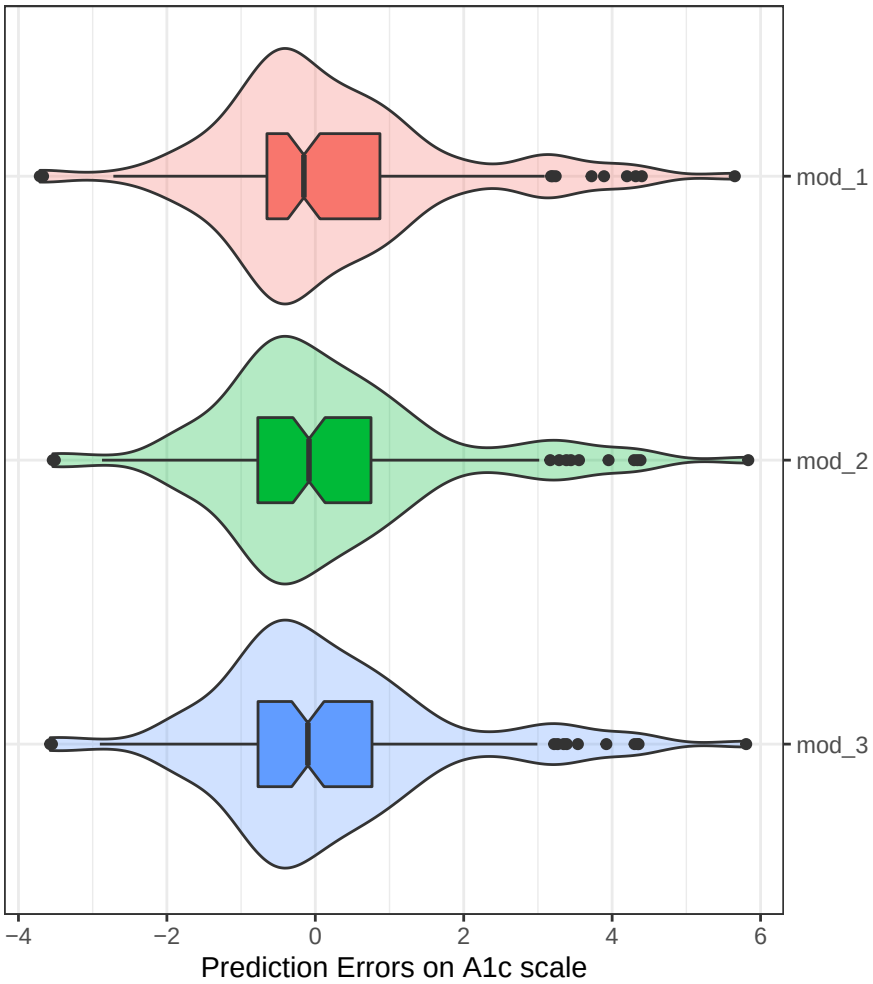


Table Comparing Model Prediction Errors

Calculate the mean absolute prediction error (MAPE), the square root of the mean squared prediction error (RMSPE) and the maximum absolute error across the predictions made by each model.

```
test_comp %>%
  group_by(name) %>%
  summarize(n = n(),
            MAPE = mean(abs(res_a1c)),
            RMSPE = sqrt(mean(res_a1c^2)),
            max_error = max(abs(res_a1c))) %>%
  kable(digits = c(0, 0, 4, 3, 2))
```

name	n	MAPE	RMSPE	max_error
mod_1	124	1.1402	1.593	5.65
mod_2	124	1.1410	1.600	5.83
mod_3	124	1.1429	1.601	5.81

Identify the largest errors

Identify the subject(s) where that maximum prediction error was made by each model, and the observed and model-fitted values of `a1c` in each case.

```
temp1 <- test_m1 %>%
  filter(abs(res_a1c) == max(abs(res_a1c)))

temp2 <- test_m2 %>%
  filter(abs(res_a1c) == max(abs(res_a1c)))

temp3 <- test_m3 %>%
  filter(abs(res_a1c) == max(abs(res_a1c)))
```

Identify the largest errors (Results)

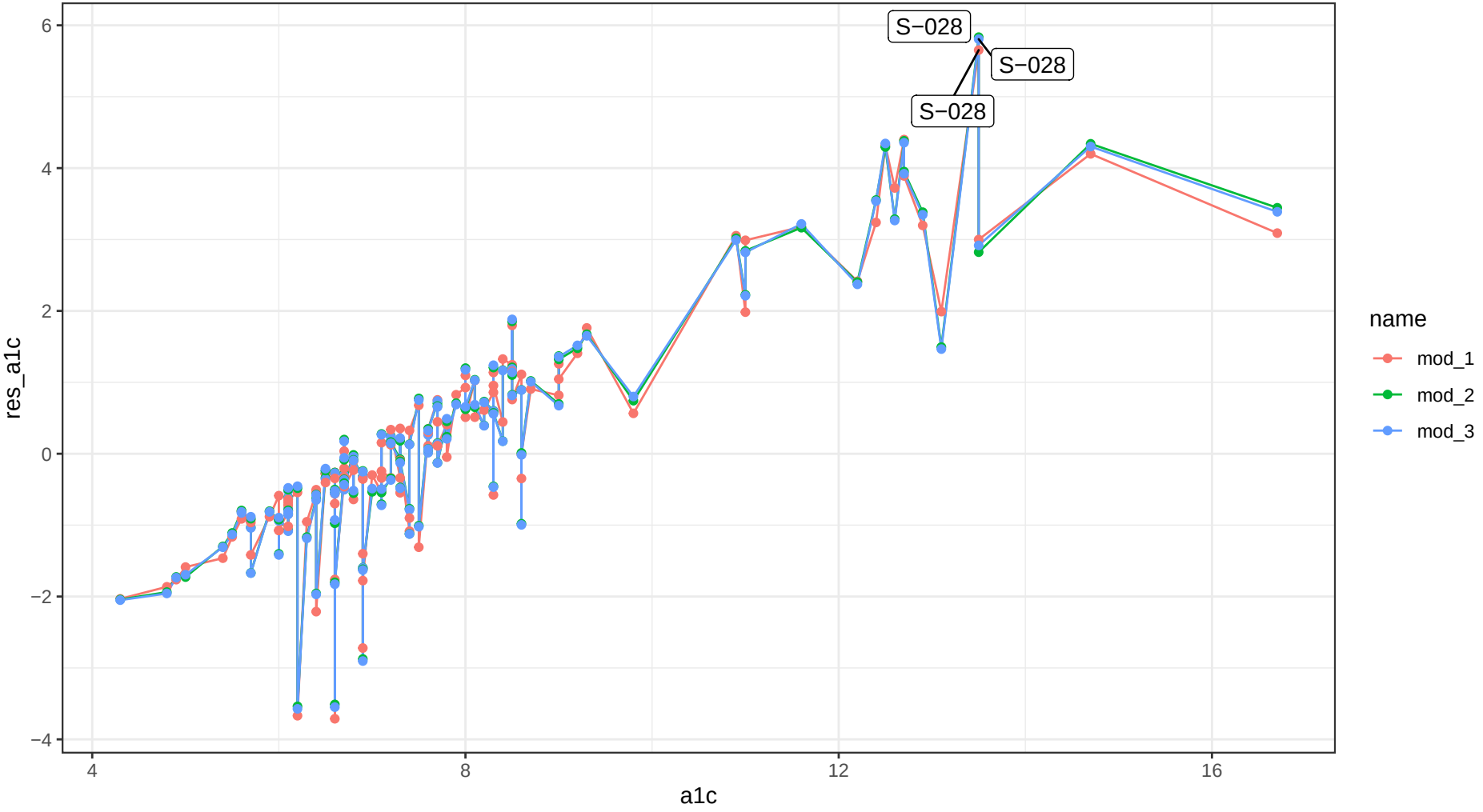
Identify the subject(s) where that maximum prediction error was made by each model, and the observed and model-fitted values of a1c in each case.

```
bind_rows(temp1, temp2, temp3) %>%
  select(subject, name, a1c, fit_a1c, res_a1c)
```

```
# A tibble: 3 x 5
  subject name    a1c fit_a1c res_a1c
  <chr>   <chr> <dbl>   <dbl>   <dbl>
1 S-028   mod_1  13.5    7.85    5.65
2 S-028   mod_2  13.5    7.67    5.83
3 S-028   mod_3  13.5    7.69    5.81
```

Line Plot of the Errors?

Compare the errors that are made at each level of observed A1c?



Code for the Line Plot of the Prediction Errors

```
ggplot(test_comp, aes(x = a1c, y = res_a1c,
                      group = name)) +
  geom_line(aes(col = name)) +
  geom_point(aes(col = name)) +
  geom_label_repel(data = test_comp %>%
                 filter(subject == "S-028"),
                 aes(label = subject))
```

What if we ignored S-028 for a moment?

All three miss this subject substantially, but without S-028, we have:

```
test_comp %>% filter(subject != "S-028") %>%
  group_by(name) %>%
  summarize(n = n(),
            MAPE = mean(abs(res_a1c)),
            RMSPE = sqrt(mean(res_a1c^2)),
            max_error = max(abs(res_a1c))) %>%
  kable(digits = c(0, 0, 4, 3, 2))
```

name	n	MAPE	RMSPE	max_error
mod_1	123	1.1035	1.516	4.40
mod_2	123	1.1029	1.518	4.38
mod_3	123	1.1050	1.520	4.36

Excluding subject S-028, the models look very comparable now.

Conclusions now?

1. In-sample model predictions are about equally accurate for each of the three models. Model 2 looks better in terms of adjusted R^2 and AIC, but model 1 looks better on BIC. There's really not much to choose from there.
2. Residual plots look similarly reasonable for linearity, Normality and constant variance in all three models.
3. Excluding a bad miss on one subject in the test sample, all three models do about equally well, with perhaps `mod_3` very slightly better according to all three metrics (MAPE, RMSPE and max_error) in the test sample.

So, what should our "most useful" model be?

Repeating our 431 Strategy

Which model is "most useful" in a prediction context?

1. Split the data into a model development (training) sample of about 70-80% of the observations, and a model test (holdout) sample, containing the remaining observations.
2. Develop candidate models using the development sample.
3. Assess the quality of fit for candidate models within the development sample.
4. Check adherence to regression assumptions in the development sample.
5. When you have candidates, assess them based on the accuracy of the predictions they make for the data held out (and thus not used in building the models.)
6. Select a "final" model for use based on the evidence in steps 3, 4 and especially 5.

OK. Let's do all of that again, but faster and using the imputed data.

Partition imputed data from dm1_imp

This time, we'll build an 80% development, 20% holdout partition of the dm1_imp data, and we'll also change our random seed, just for fun.

```
set.seed(2020999)

dm1_imp_train <- dm1_imp %>%
  slice_sample(prop = 0.8, replace = FALSE)

dm1_imp_test <-
  anti_join(dm1_imp, dm1_imp_train, by = "subject")

dim(dm1_imp_train); dim(dm1_imp_test)
```

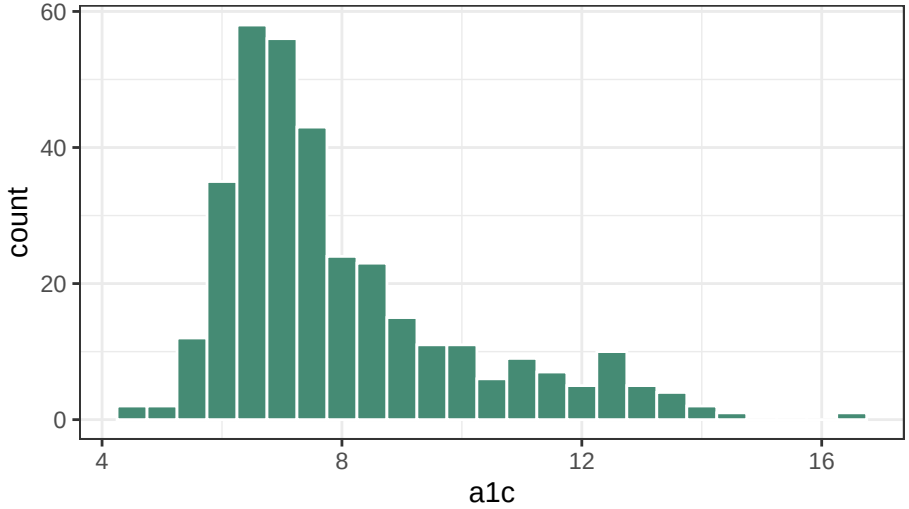
```
[1] 342   5
```

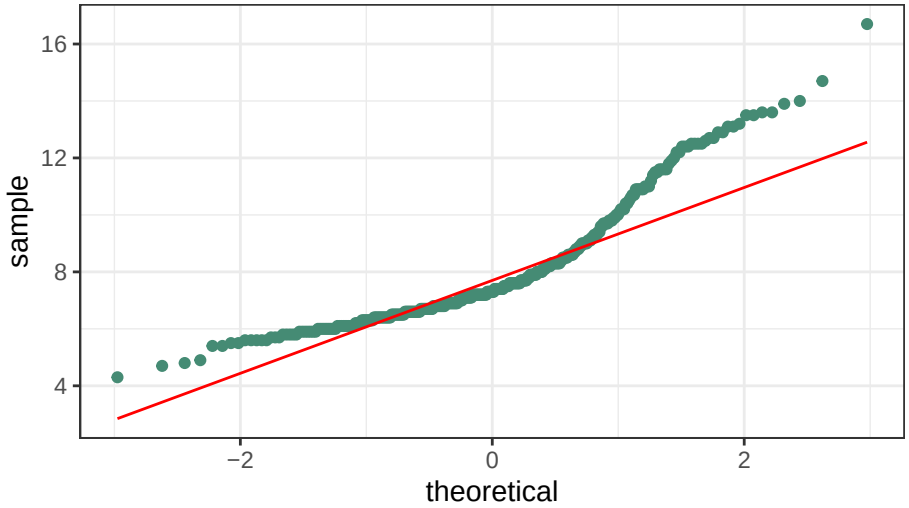
```
[1] 86  5
```

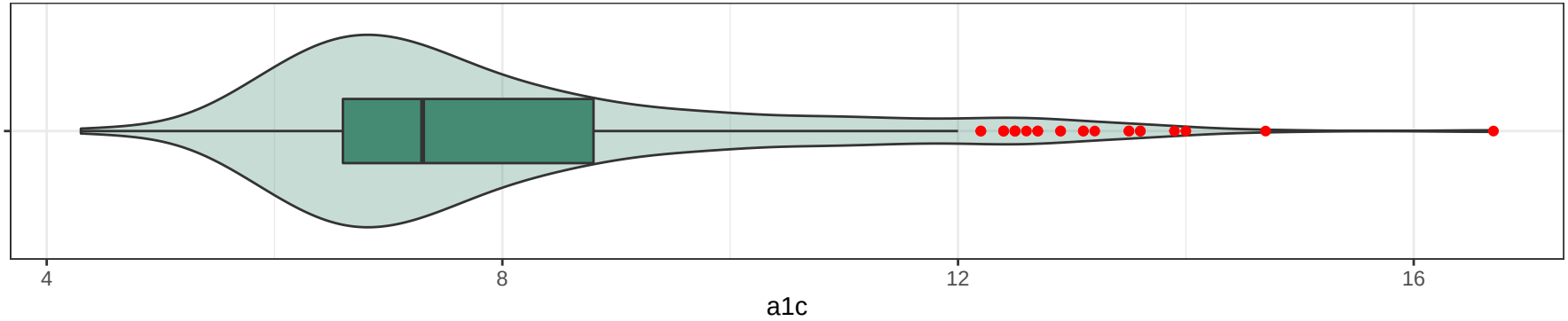
Distribution of `a1c` in training sample

Hemoglobin A1c values (%)

Model Development Sample after imputation: 342 adults with diabetes



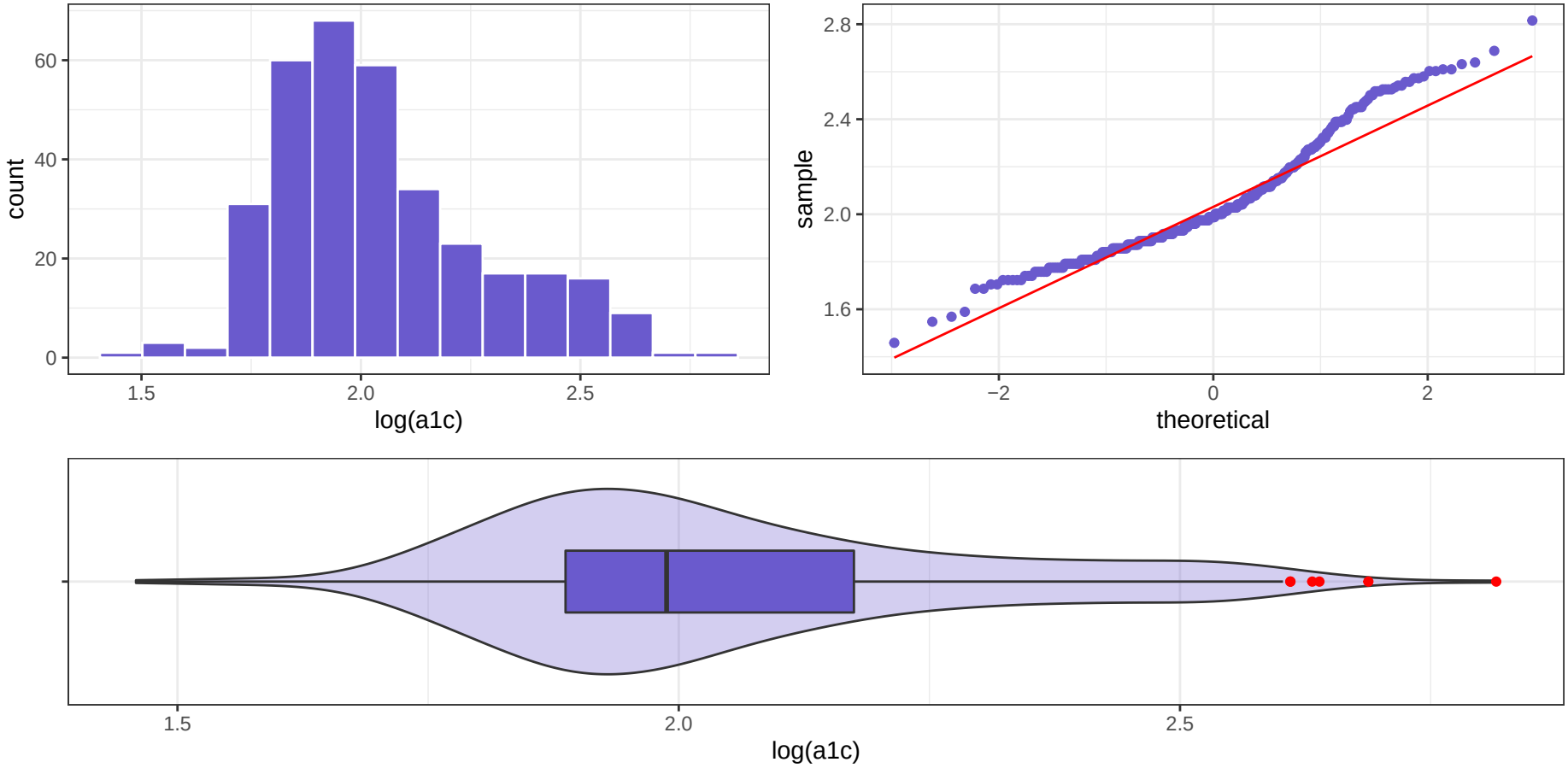




Consider a log transformation?

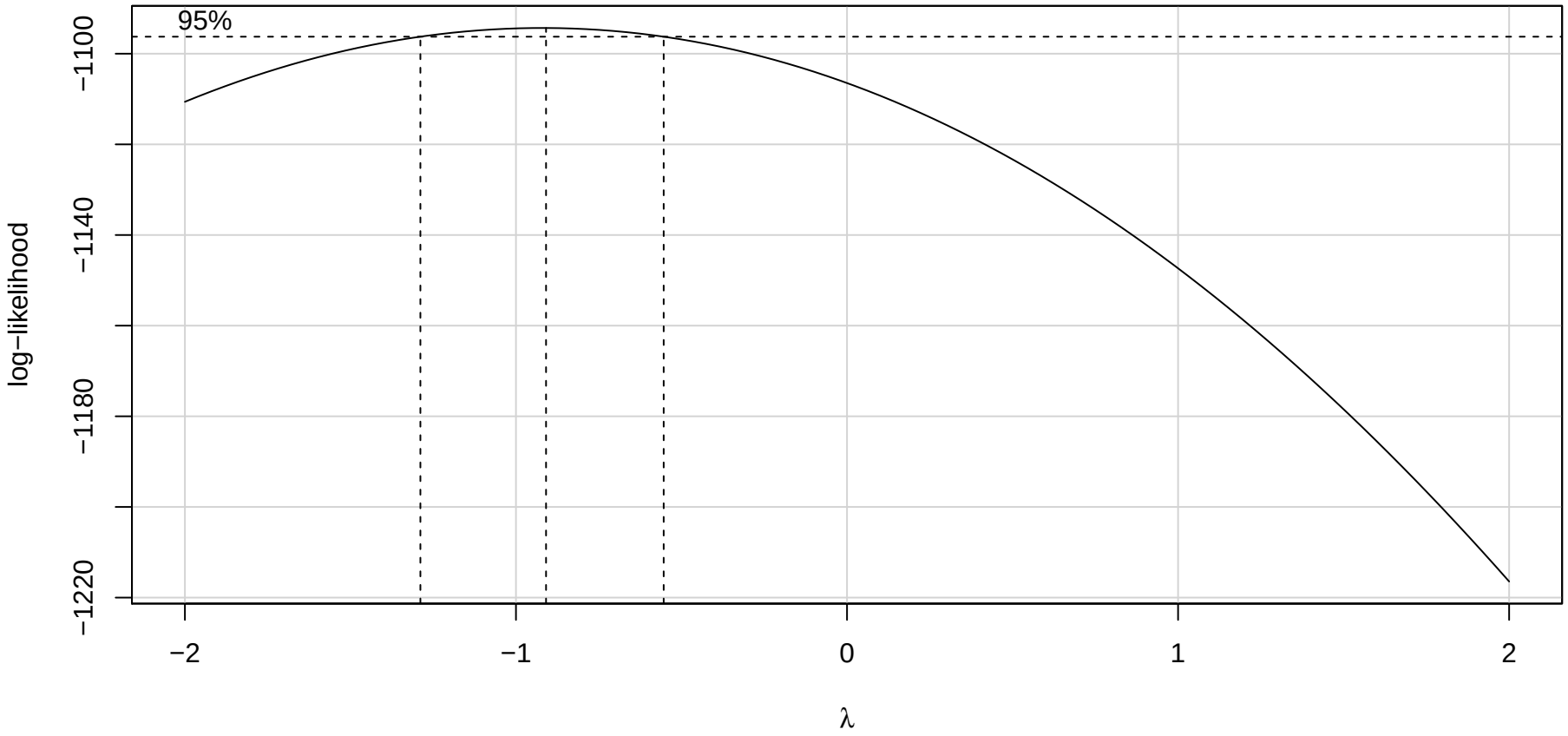
Natural Logarithm of Hemoglobin A1c

Model Development Sample: 342 adults with diabetes



What does Box-Cox suggest?

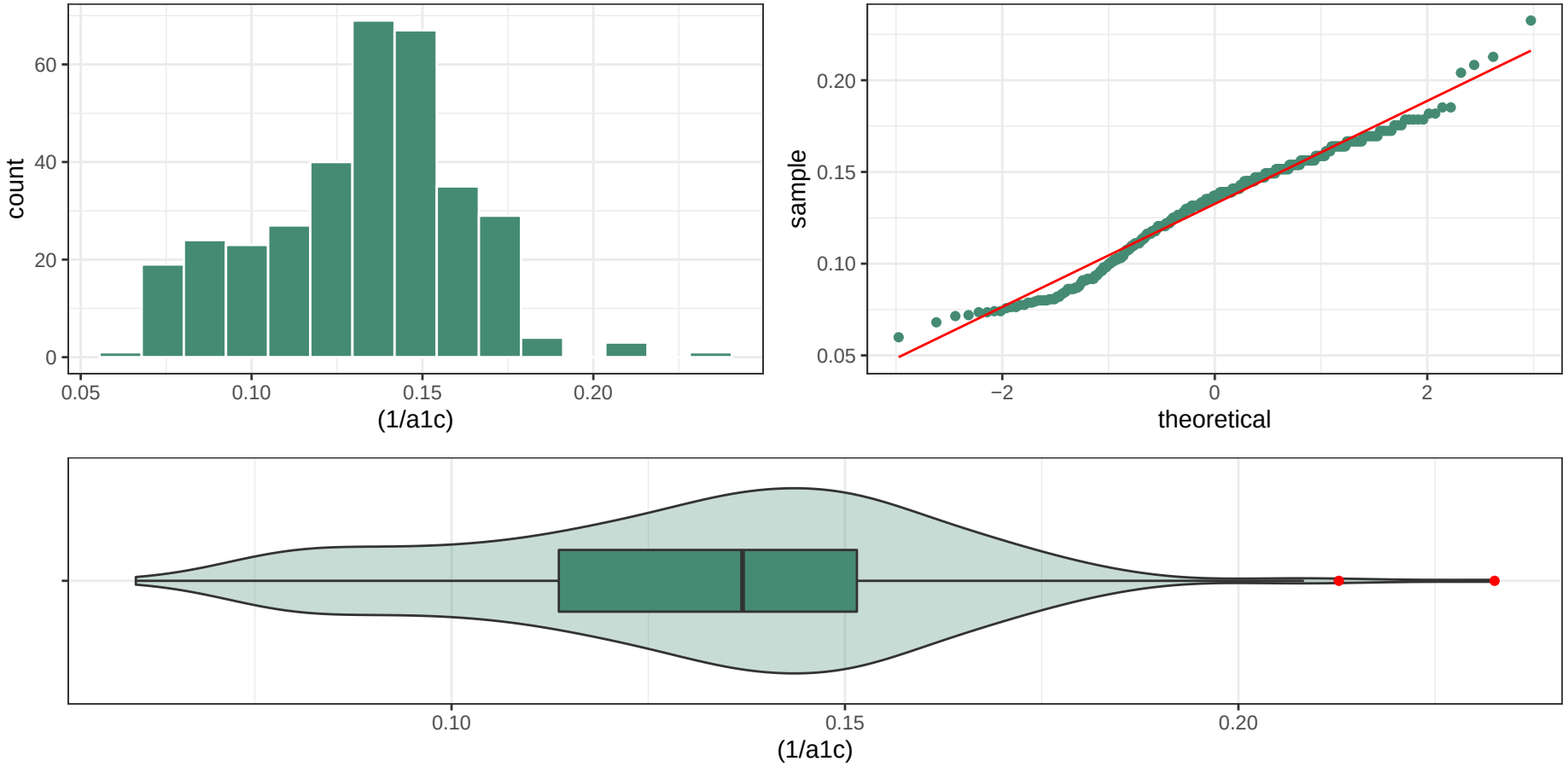
```
imod_0 <- lm(a1c ~ a1c_old + age + income,
             data = dm1_imp_train)
boxCox(imod_0)
```



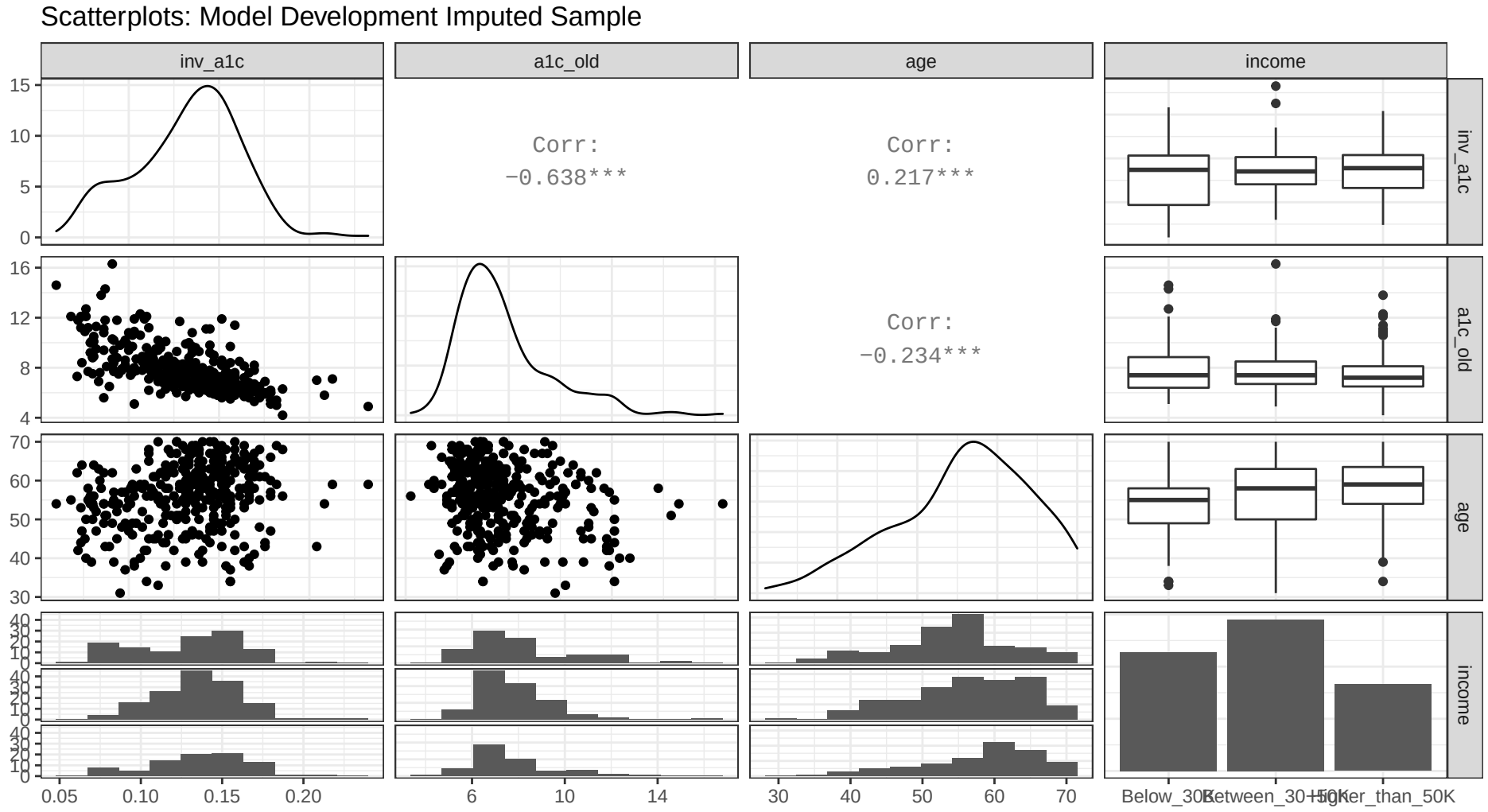
Inverse of A1c again?

Inverse of Hemoglobin A1c

Model Development Sample after Imputation: 342 adults with diabetes



Scatterplot Matrix (code on next slide)



Fitting the Same Three Models

- Remember we're using the model development sample here.

```
imod_1 <- lm((1/a1c) ~ a1c_old, data = dm1_imp_train)

imod_2 <- lm((1/a1c) ~ a1c_old + age, data = dm1_imp_train)

imod_3 <- lm((1/a1c) ~ a1c_old + age + income,
             data = dm1_imp_train)
```

Assess the quality of fit for candidate models within the development sample.

Tidied coefficients (imod_1)

```
tidy_im1 <- tidy(imod_1, conf.int = TRUE, conf.level = 0.95)

tidy_im1 %>%
  select(term, estimate, std.error, p.value,
         conf.low, conf.high) %>%
  knitr::kable(digits = 4)
```

term	estimate	std.error	p.value	conf.low	conf.high
(Intercept)	0.2121	0.0053	0	0.2016	0.2226
a1c_old	-0.0102	0.0007	0	-0.0115	-0.0089

The Regression Equation (imod_1)

Again, we'll use the equatiomatic package.

```
extract_eq(imod_1, use_coefs = TRUE, coef_digits = 4,
           ital_vars = TRUE, wrap = TRUE, terms_per_line = 3)
```

$$(1/a1c) = 0.2121 - 0.0102(a1c_old) + \epsilon$$

Summary of Fit Quality (imod_1)

```
glance(imod_1) %>%
  mutate(name = "imod_1") %>%
  select(name, r.squared, adj.r.squared,
         sigma, AIC, BIC) %>%
  knitr::kable(digits = c(0, 3, 3, 3, 0, 0))
```

name	r.squared	adj.r.squared	sigma	AIC	BIC
imod_1	0.408	0.406	0.022	-1627	-1615

Tidied coefficients (imod_2)

```
tidy_im2 <- tidy(imod_2, conf.int = TRUE, conf.level = 0.95)

tidy_im2 %>%
  select(term, estimate, std.error, p.value,
         conf.low, conf.high) %>%
  knitr::kable(digits = 4)
```

term	estimate	std.error	p.value	conf.low	conf.high
(Intercept)	0.1962	0.0108	0.0000	0.1749	0.2175
a1c_old	-0.0100	0.0007	0.0000	-0.0113	-0.0086
age	0.0002	0.0001	0.0932	0.0000	0.0005

The Regression Equation (imod_2)

Again, we'll use the `equatiomatic` package, and **results = 'asis'**.

```
extract_eq(imod_2, use_coefs = TRUE, coef_digits = 4,
           ital_vars = TRUE)
```

$$(1/a1c) = 0.1962 - 0.01(a1c_old) + 2e - 04(age) + \epsilon$$

Summary of Fit Quality (imod_2)

```
glance(imod_2) %>%
  mutate(name = "imod_2") %>%
  select(name, r.squared, adj.r.squared,
         sigma, AIC, BIC) %>%
  knitr::kable(digits = c(0, 3, 3, 3, 0, 0))
```

name	r.squared	adj.r.squared	sigma	AIC	BIC
imod_2	0.412	0.409	0.022	-1627	-1612

Tidied coefficients (imod_3)

```
tidy_im3 <- tidy(imod_3, conf.int = TRUE, conf.level = 0.95)

tidy_im3 %>%
  select(term, estimate, se = std.error,
         low = conf.low, high = conf.high, p = p.value) %>%
  knitr::kable(digits = c(4,4,4,4,3))
```

term	estimate	se	low	high	p
(Intercept)	0.1948	0.0109	0.1735	0.216	0.0000
a1c_old	-0.0099	0.0007	-0.0113	-0.009	0.0000
age	0.0002	0.0001	-0.0001	0.001	0.1382
incomeBetween_30-50K	0.0045	0.0028	-0.0010	0.010	0.1120
incomeHigher_than_50K	0.0026	0.0033	-0.0038	0.009	0.4251

The Regression Equation (imod_3)

Again, we'll use the equatiomatic package.

```
extract_eq(imod_3, use_coefs = TRUE, coef_digits = 4,
           ital_vars = TRUE, wrap = TRUE, terms_per_line = 2)
```

$$
\begin{aligned}
(1/a1c) = 0.1948 - 0.0099(a1c_old) + \\
2e-04(age) + 0.0045(income_{Between_30-50K}) + \\
0.0026(income_{Higher_than_50K}) + \epsilon
\end{aligned}
$$

Summary of Fit Quality (imod_3)

```
glance(imod_3) %>%
  mutate(name = "imod_3") %>%
  select(name, r.squared, adj.r.squared,
         sigma, AIC, BIC) %>%
  knitr::kable(digits = c(0, 3, 3, 3, 0, 0))
```

name	r.squared	adj.r.squared	sigma	AIC	BIC
imod_3	0.417	0.41	0.022	-1626	-1603

I checked stepwise regression again

- Even though variable selection **never** works, it is seductive.

What if we do forward selection in this situation?

```
min.model <- lm(a1c ~ 1, data = dm1_imp_train)
fwd.model <- step(min.model, direction = "forward",
                  scope = ~ a1c_old + age + income)
```

```
Start:  AIC=501.98
a1c ~ 1

          Df Sum of Sq     RSS    AIC
+ a1c_old  1    625.12  850.37 315.51
+ age      1     76.05 1399.44 485.88
+ income   2     43.97 1431.52 495.63
<none>                1475.49 501.98

Step:  AIC=315.51
```

Stepwise Regression Results

We wind up back at the model with all three predictors in this case.

```
fwd.model$coefficients
```

```
          (Intercept)                 a1c_old
           3.58233459              0.72380568
 incomeBetween_30-50K incomeHigher_than_50K
          -0.58229971             -0.33157108
                  age
          -0.01619949
```

- There is an immense amount of evidence that variable selection causes severe problems in estimation and inference.
- Stepwise regression, in particular, is an egregiously bad choice, as we'll show in 432.

Which Model Looks Best In-Sample?

For each of these summaries, which model looks best in the training sample?

model	vars	r2	adj_r2	sigma	AIC	BIC
imod_1	a1c_old	0.408	0.406	0.02231	-1626.5	-1615
imod_2	+ age	0.412	0.409	0.02225	-1627.4	-1612
imod_3	+ income	0.417	0.410	0.02223	-1625.9	-1603

What are appropriate conclusions now?

- One of the models beats the others (though not by much,) right?

Using `augment` to add fits, residuals, etc.

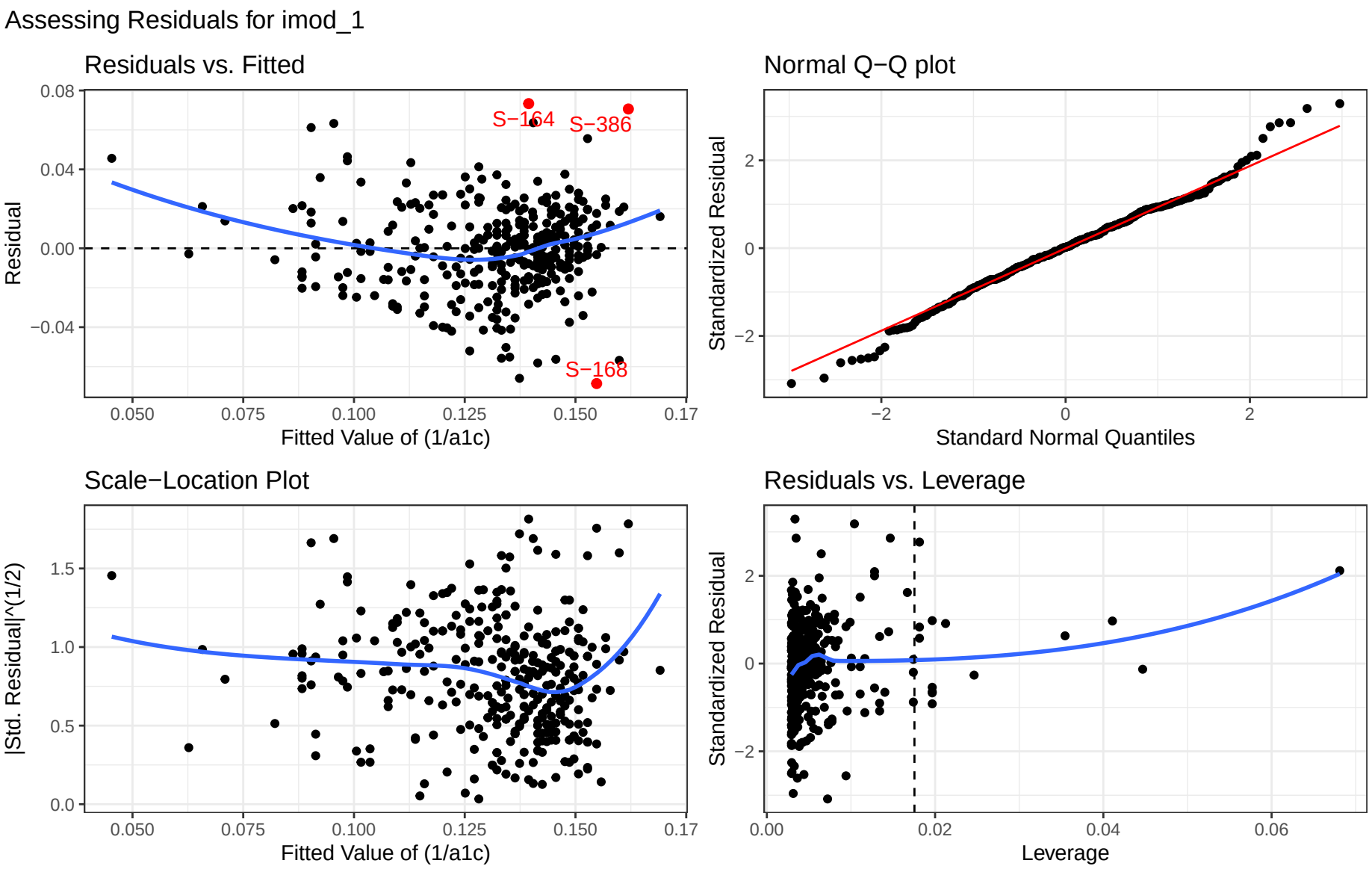
```
augi1 <- augment(imod_1, data = dm1_imp_train) %>%
  mutate(inv_a1c = 1/a1c) # add in our model's outcome

augi2 <- augment(imod_2, data = dm1_imp_train) %>%
  mutate(inv_a1c = 1/a1c) # add in our model's outcome

augi3 <- augment(imod_3, data = dm1_imp_train) %>%
  mutate(inv_a1c = 1/a1c) # add in our model's outcome
```

Checking Regression Assumptions

Four key assumptions we need to think about:

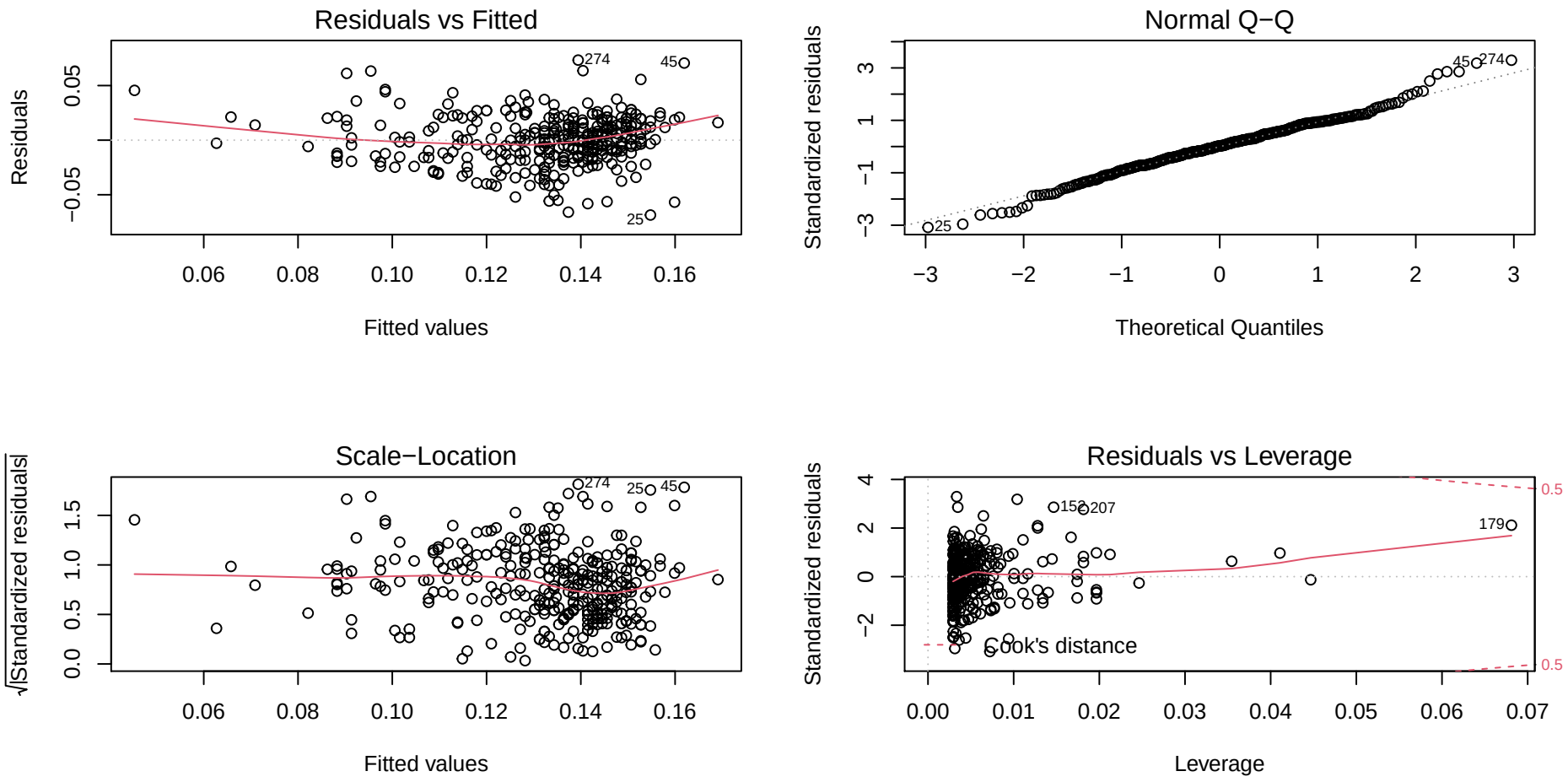
1. Linearity
2. Constant Variance (Homoscedasticity)
3. Normality
4. Independence

Main 4 Residual Plots for `imod_1` (via `ggplot2`)

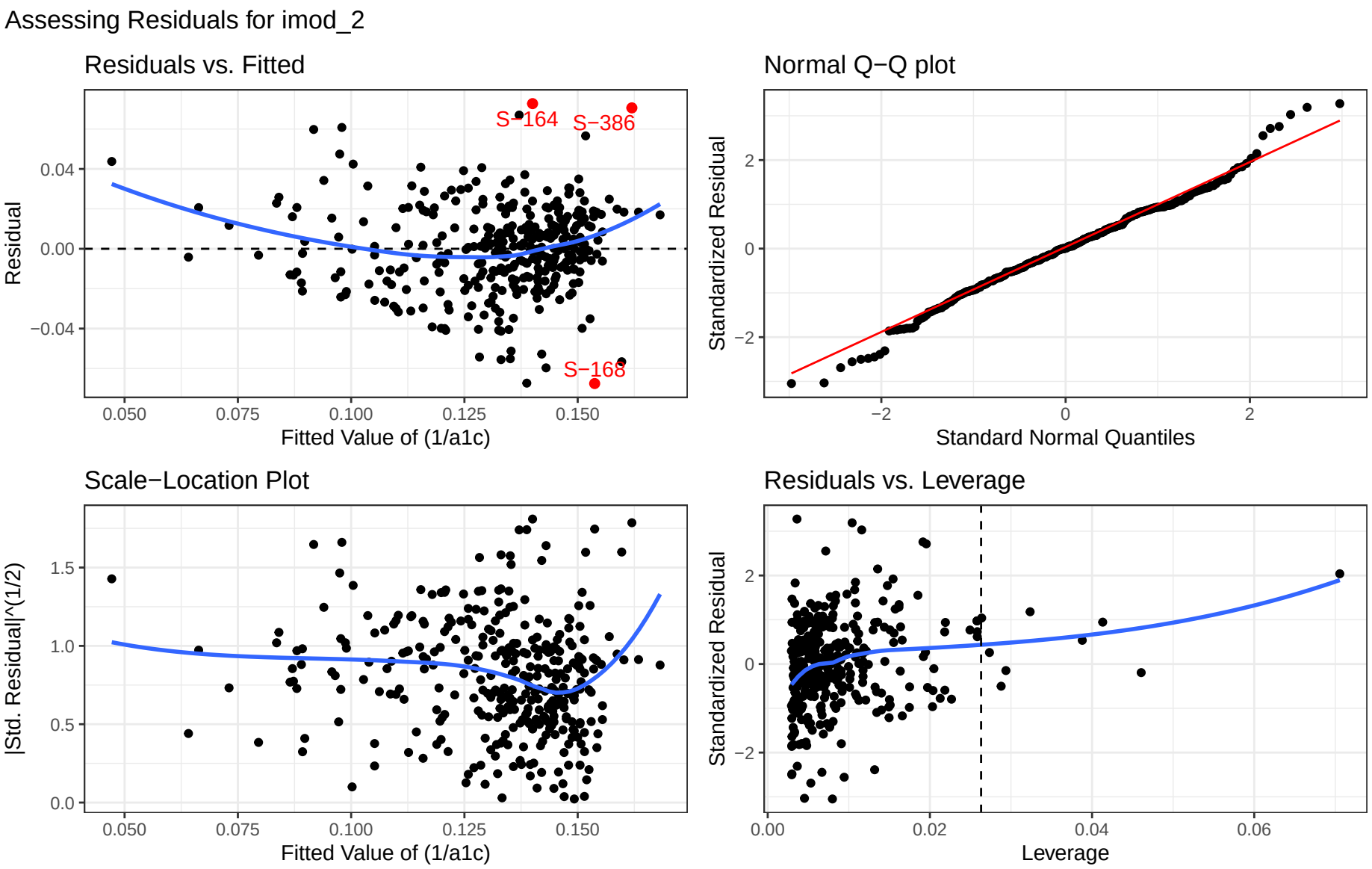


Base R Residual Plots for Model `imod_1`

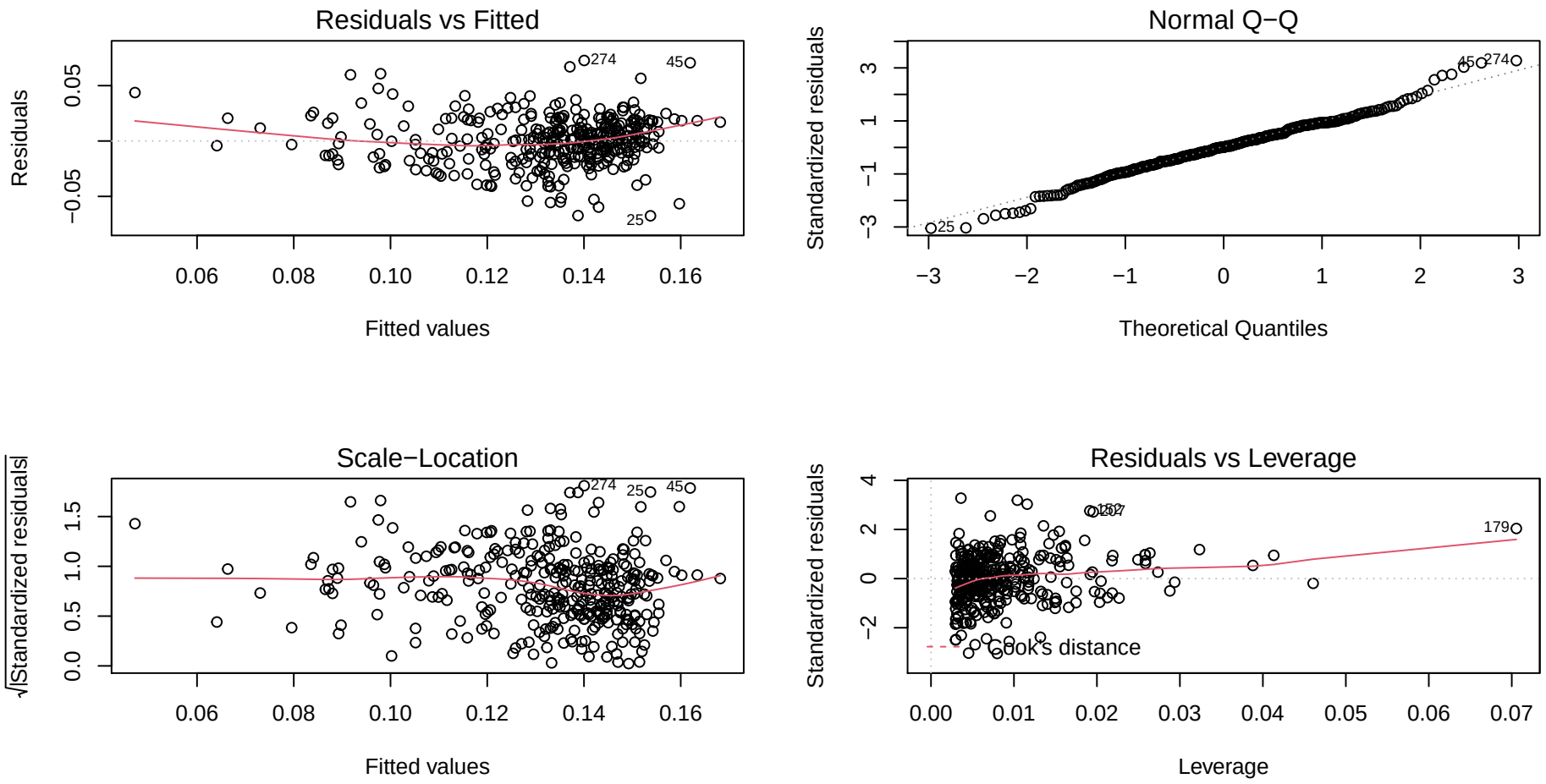
```
par(mfrow = c(2,2)); plot(imod_1); par(mfrow = c(1,1))
```



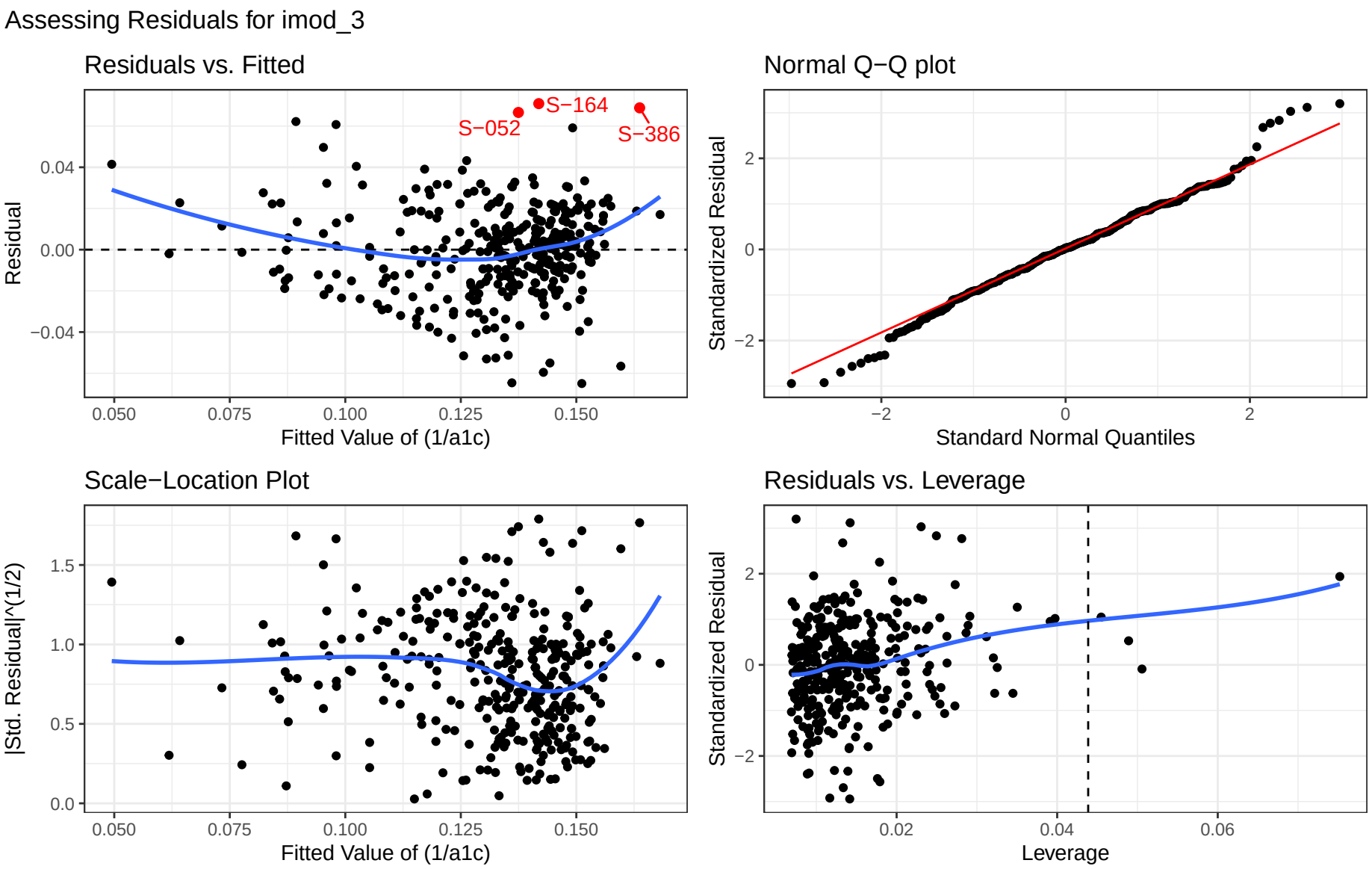
Main 4 Residual Plots for `imod_2` (via `ggplot2`)



Base R Residual Plots for Model `imod_2`

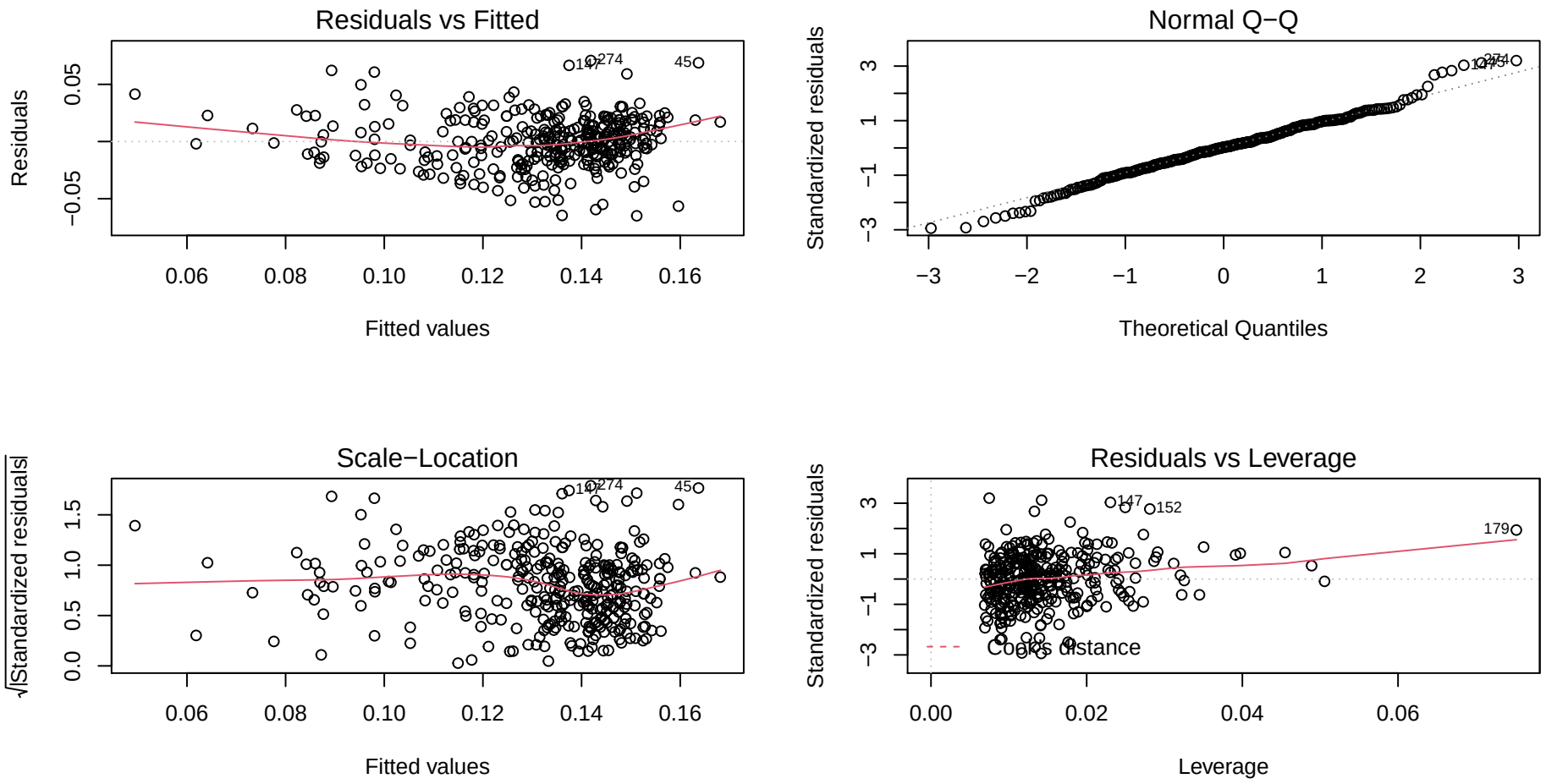


Main 4 Residual Plots for `imod_3` (via `ggplot2`)



If applicable, Cook's d >= 0.5 shown in red in bottom right plot.

Base R Residual Plots for Model `imod_3`



Is collinearity a serious issue here?

```
car::vif(imod_3)
```

```
            GVIF Df GVIF^(1/(2*Df))
a1c_old 1.059820  1        1.029475
age     1.088064  1        1.043103
income  1.036183  2        1.008926
```

None of these values exceed 5, so it doesn't seem like there's any problem.

```
car::vif(imod_2)
```

```
 a1c_old      age
1.057711 1.057711
```

Conclusions so far?

We're getting about the same story as we did with the complete cases, right?

1. In-sample model predictions are about equally accurate for each of the three models. Model 3 (with all three predictors) looks a little better on all summaries (adjusted R^2, AIC and BIC, in particular.)
2. Residual plots look similarly reasonable for linearity, Normality and constant variance in all three models.

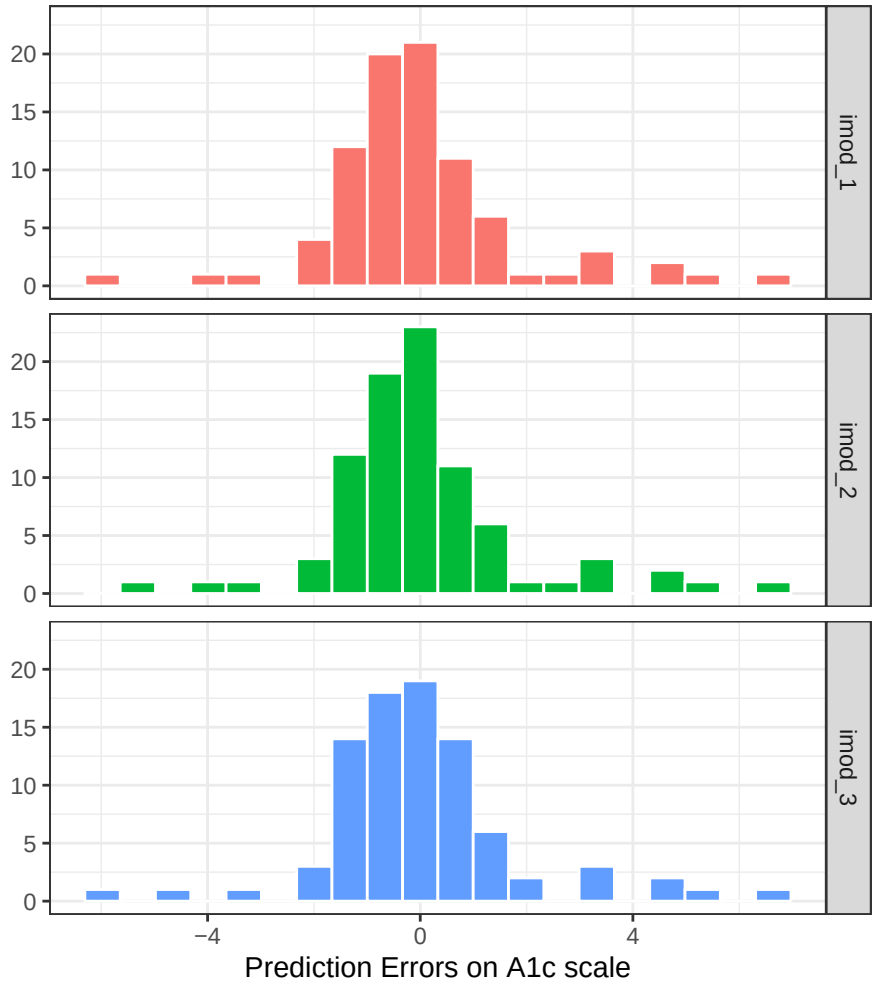
Calculate prediction errors in test samples

```
test_im1 <- augment(imod_1, newdata = dm1_imp_test) %>%
  mutate(name = "imod_1", fit_a1c = 1 / .fitted,
         res_a1c = a1c - fit_a1c)

test_im2 <- augment(imod_2, newdata = dm1_imp_test) %>%
  mutate(name = "imod_2", fit_a1c = 1 / .fitted,
         res_a1c = a1c - fit_a1c)

test_im3 <- augment(imod_3, newdata = dm1_imp_test) %>%
  mutate(name = "imod_3", fit_a1c = 1 / .fitted,
         res_a1c = a1c - fit_a1c)
```

Combine test sample results from the three models

```
test_icomp <- bind_rows(test_im1, test_im2, test_im3) %>%
  arrange(subject, name)
```

Visualize Test-Sample Prediction Errors



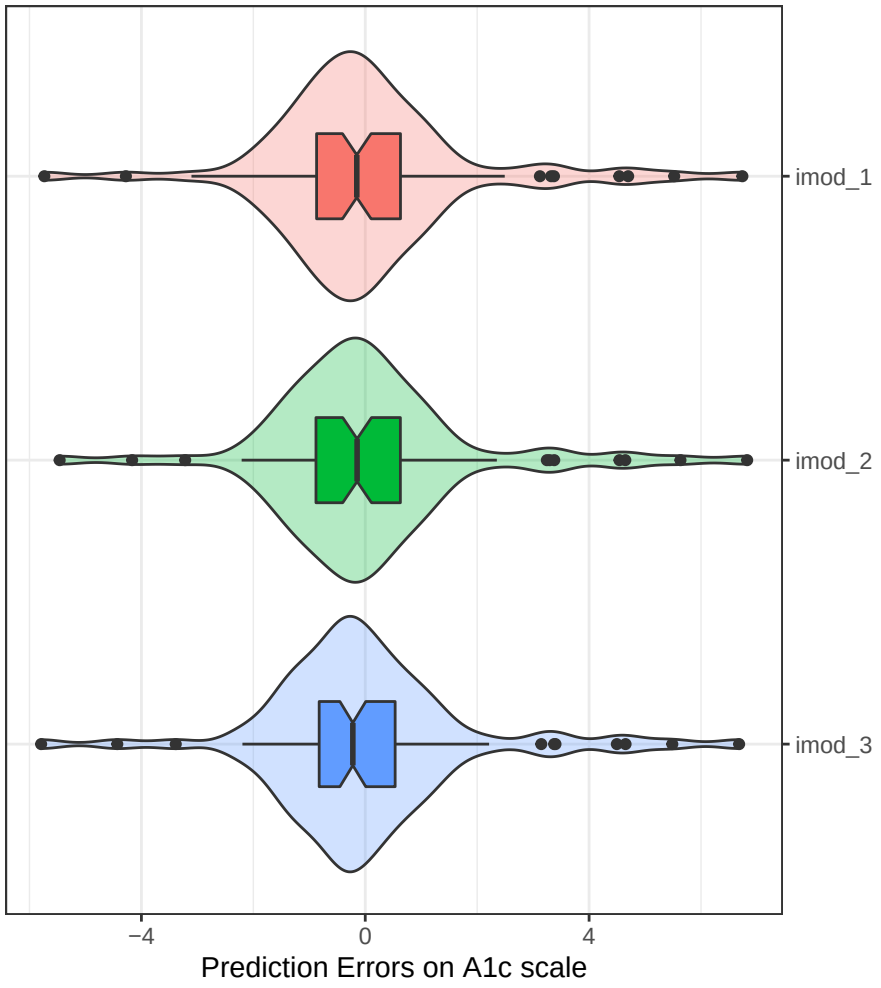


Table Comparing Model Prediction Errors

Calculate the mean absolute prediction error (MAPE), the square root of the mean squared prediction error (RMSPE) and the maximum absolute error across the predictions made by each model.

```
test_icomp %>%
  group_by(name) %>%
  summarize(n = n(),
            MAPE = mean(abs(res_a1c)),
            RMSPE = sqrt(mean(res_a1c^2)),
            max_error = max(abs(res_a1c))) %>%
  kable(digits = c(0, 0, 3, 3, 2))
```

name	n	MAPE	RMSPE	max_error
imod_1	86	1.196	1.809	6.74
imod_2	86	1.192	1.803	6.82
imod_3	86	1.209	1.818	6.68

Identify the largest errors (Results)

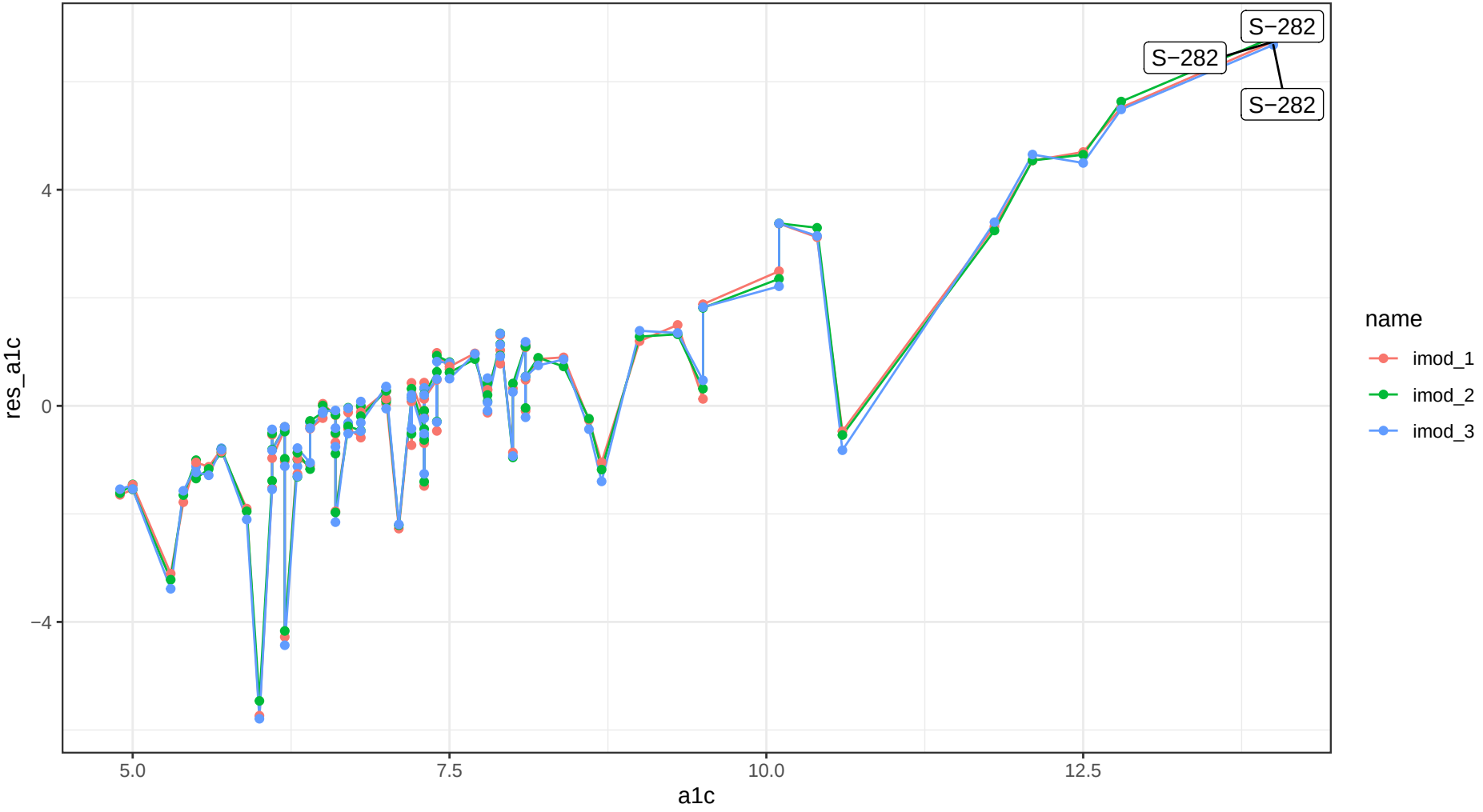
Identify the subject(s) where that maximum prediction error was made by each model, and the observed and model-fitted values of a1c in each case.

```
bind_rows(tempi1, tempi2, tempi3) %>%
  select(subject, name, a1c, fit_a1c, res_a1c)
```

```
# A tibble: 3 x 5
  subject name     a1c fit_a1c res_a1c
  <chr>   <chr>  <dbl>   <dbl>   <dbl>
1 S-282   imod_1    14    7.26    6.74
2 S-282   imod_2    14    7.18    6.82
3 S-282   imod_3    14    7.32    6.68
```

Line Plot of the Errors?

Compare the errors that are made at each level of observed A1c?



What if we ignored S-282 for a moment?

All three miss this subject substantially, but without S-282, we have:

```
test_icomp %>% filter(subject != "S-282") %>%
  group_by(name) %>%
  summarize(n = n(),
            MAPE = mean(abs(res_a1c)),
            RMSPE = sqrt(mean(res_a1c^2)),
            max_error = max(abs(res_a1c))) %>%
  kable(digits = c(0, 0, 3, 2, 2))
```

name	n	MAPE	RMSPE	max_error
imod_1	85	1.131	1.67	5.74
imod_2	85	1.126	1.66	5.63
imod_3	85	1.145	1.68	5.79

Excluding subject S-282, the models look pretty comparable, but `imod_2` is maybe a little better.

Key Summaries

With complete cases,

- in-sample: all three models look OK on assumptions in residual plots, model 2 looks like it fits a little better by Adjusted R^2 and AIC, model 1 looks slightly better by BIC.
- out-of-sample: distributions of errors are similar. Model 1 has lowest MAPE, RMPSE and maximum error, but all three models are pretty close to each other.

With imputation,

- in-sample: nothing disastrous in residual plots, model 3 looks a little better than the others on adjusted R^2, AIC and BIC.
- out-of-sample: Model 2 has the smallest MAPE, RMSE, while model 3 has the smallest maximum predictive error, but if we drop the one worst-fit case, then model 2 sweeps all three categories.

So what can we conclude? Does this particular imputation strategy have a big impact?

Again, this is our 431 Strategy

Which model is "most useful" in a prediction context?

1. Split the data into a model development (training) sample of about 70-80% of the observations, and a model test (holdout) sample, containing the remaining observations.
2. Develop candidate models using the development sample.
3. Assess the quality of fit for candidate models within the development sample.
4. Check adherence to regression assumptions in the development sample.
5. When you have candidates, assess them based on the accuracy of the predictions they make for the data held out (and thus not used in building the models.)
6. Select a "final" model for use based on the evidence in steps 3, 4 and especially 5.

Coming Up

Happy Thanksgiving!

Good luck with Project B. You should now be prepared to do the whole thing, assuming I've signed off on your Study 2 Data Plan.

- Next Class: 2020-12-01.